

Recommendation Systems: Placement, Research, and Engineering Guide

Recommendation systems estimate which items a user is likely to value and decide which of millions of candidates to show. Production systems usually form a pipeline: candidate generation retrieves hundreds of plausible items, ranking orders them, re-ranking applies diversity and business constraints, and feedback updates future models. This guide treats each major approach independently while showing how the pieces connect.

Notation used throughout: users are u , items are i , observed interactions are r_{ui} , predicted relevance is $\hat{r}_{ui}$, latent dimension is k , and a ranked list has length K . An interaction may be an explicit rating or an implicit event such as a click, watch, save, purchase, or skip.

Popularity-Based Recommendation

1. Overview

Popularity-based recommendation ranks items using aggregate behavior rather than personal history. Typical signals are views, purchases, ratings, completion rate, or recent engagement. It is useful as a baseline, a fallback for anonymous/new users, and a source of trending candidates. News homepages, app-store charts, and “most watched” rails commonly use it.

2. Intuition

It is the digital equivalent of a bookstore’s “best sellers” table. If nothing is known about a visitor, collective demand is a reasonable prior. A better system than raw counts also asks whether an item is recent, well-rated with enough evidence, and popular within the user’s region or context.

3. Prerequisites

- Pandas aggregation and sorting
- Counts, means, priors, and Bayesian smoothing
- Time decay and categorical segmentation
- Offline train/validation/test splits

4. Core Concepts

- **Global popularity:** one score for every item. It matters as a simple, strong baseline. Example: top purchased products. Interview angle: it is non-personalized.
- **Segmented popularity:** aggregate by country, device, age band, or category. It adds coarse personalization. Example: trending songs in India. Interview angle: keep segments large enough for stable estimates.
- **Time-aware popularity:** discount old events so trends react quickly. Example: breaking news. Interview angle: discuss the freshness-stability trade-off.

- **Quality-adjusted popularity:** combine volume with rating, conversion, or completion. It prevents clickbait from winning on clicks alone. Interview angle: explain Bayesian shrinkage.

5. Algorithm / Working Process

1. Collect interactions before a cutoff time.
2. Assign event weights; for example, view 1, save 3, purchase 5.
3. Optionally apply time decay and segment filters.
4. Aggregate weighted events per item.
5. Smooth noisy averages and remove unavailable/unsafe items.
6. Sort by score and return the top K .

Input is an event table; output is an ordered item list. There is usually no gradient-based training. Inference is a fast lookup from a periodically refreshed table.

6. Mathematical Foundation

Raw popularity is $p_i = \sum_u c_{ui}$, where c_{ui} is a count or event weight. Exponential decay gives

$$p_i(t) = \sum_e w_e \exp(-\lambda(t - t_e)),$$

where half-life $h = \ln(2)/\lambda$. For average rating R_i from v_i votes, shrink toward global mean C :

$$\text{score}_i = (v_i/(v_i+m)) R_i + (m/(v_i+m)) C.$$

m controls how much evidence is needed before an item escapes the prior. A common alternative for positive/negative votes is the lower bound of a Wilson confidence interval.

7. Practical Implementation

```
import numpy as np
import pandas as pd

def trending(events: pd.DataFrame, now: pd.Timestamp, half_life_days=7, k=10):
    """events columns: item_id, event, timestamp"""
    weight = {"view": 1.0, "save": 3.0, "purchase": 5.0}
    age_days = (now - pd.to_datetime(events["timestamp"])).dt.total_seconds() /
    86400
    decay = np.exp(-np.log(2) * age_days / half_life_days)
    scored = events.assign(score=events["event"].map(weight).fillna(0) * decay)
    return (scored.groupby("item_id")["score"].sum()
            .sort_values(ascending=False).head(k))

events = pd.DataFrame({
    "item_id": [1, 1, 2, 2, 3],
    "event": ["view", "purchase", "view", "save", "purchase"],
    "timestamp": pd.to_datetime(["2026-08-11", "2026-08-12", "2026-08-01",
                                "2026-08-10", "2026-07-01"]),
})
print(trending(events, pd.Timestamp("2026-08-12")))
```

8. Code Explanation

Event weights express unequal value, while exponential decay makes a seven-day-old event contribute half as much. **grouphy** builds one score per item and sorting produces the recommendation list. In production, compute this incrementally and filter already consumed or unavailable items at serving time.

9. Training / Evaluation

Use chronological splits: build popularity from past events and test whether later interactions contain recommended items. Evaluate Recall@K, NDCG@K, coverage, novelty, and business outcomes. Tune event weights, half-life, and segmentation on validation data. A long half-life is stable but slow; a short one is responsive but noisy.

10. Complexity and Cost

Aggregation is $O(E)$ for E events; selecting top items is $O(I \log K)$ with a heap or $O(I \log I)$ by full sorting. Storage is $O(I)$ per segment. CPU batch or streaming jobs are sufficient; online lookup is near $O(K)$.

11. Common Use Cases

Trending news, best sellers, popular videos, new-user onboarding, fallback recommendations, regional charts, and candidate generation for a personalized ranker.

12. Common Mistakes

- Using future events when constructing historical popularity
- Ranking by mean rating with only one or two votes
- Ignoring item age, exposure bias, availability, bots, or repeated events
- Evaluating only clicks and rewarding clickbait
- Calling a segmented top list fully personalized

13. Edge Cases / Limitations

It creates rich-get-richer feedback, gives low catalog coverage, cannot capture individual taste, and can suppress new items. Sudden bot activity can corrupt counts. Global trends may be irrelevant or unsafe for a local audience.

14. Variations

- **Trending score:** adds decay; use for news and short-form media; placement-important.
- **Bayesian/Wilson ranking:** adjusts for uncertainty; use with ratings or votes; important in projects.
- **Contextual popularity:** separate lists by time, location, or category; common in production.
- **Exploration-aware lists:** reserve slots for new items; important for research and marketplaces.

15. Related Topics

Popularity is the prior used by cold-start systems. Content-based models add item relevance, collaborative filtering adds behavioral personalization, and bandits correct the exploit-only behavior by exploring uncertain items.

16. Interview Questions

1. **Why is popularity a strong baseline?** Collective behavior has high signal and the method has low variance.
2. **Is it personalized?** Not globally; segmentation provides only coarse personalization.
3. **How do you handle trends?** Apply time decay or compute scores in rolling windows.
4. **Why not rank by average rating?** Small samples produce unstable extreme means.
5. **What is Bayesian shrinkage?** Pull uncertain item estimates toward a global prior.
6. **How do you evaluate it?** Use a temporal holdout and top-K ranking metrics.
7. **What is its main bias?** Exposure reinforces already popular items.
8. **How do new items appear?** Editorial rules, exploration slots, or content-based candidates.
9. **When should event types be weighted?** When events represent different user value or intent.
10. **Batch or online computation?** Usually batch/stream aggregation plus cached online lookup.

17. Practice Tasks

Implement raw and decayed popularity; compare them on MovieLens timestamps; tune the half-life by Recall@10; detect duplicate/bot events; add country-level lists with a global fallback.

18. Project Ideas

Project	What it does	Stack and data	Resume value
News trend engine	Ranks stories with decay	Pandas/FastAPI; MIND news	Streaming features and freshness
Marketplace charts	Builds category best sellers	Python/SQL; Retailrocket	Segmentation and leakage-safe evaluation
Music discovery rail	Blends popularity and novelty	Pandas; Last.fm	Multi-objective recommendation

19. Quick Revision

- **Key idea:** recommend aggregate winners when personal signal is absent.
- **Main formula:** weighted, time-decayed event sum.
- **Use:** baseline, fallback, trending candidates.
- **Metrics:** Recall@K, NDCG@K, coverage, novelty.
- **Trap:** future leakage and popularity feedback loops.
- **One-liner:** popularity is a robust prior, not true personalization.

20. Final Cheat Sheet

Item	Summary
Input/output	Interaction log -> top-K items
Steps	Weight, decay, aggregate, filter, rank
Hyperparameters	Event weights, half-life, prior strength, segment

Item	Summary
Pros	Simple, fast, explainable, strong fallback
Cons	Non-personalized, biased, weak discovery
Best use	Cold users, trends, candidate fallback

Content-Based Filtering

1. Overview

Content-based filtering recommends items whose attributes resemble items a user liked. Features may be genres, tags, text embeddings, image features, price, creator, or structured metadata. It is useful when item descriptions are rich, interactions are sparse, or new items must be recommended immediately. Job matching, news, e-commerce, and document recommendation rely heavily on it.

2. Intuition

If a user reads several “Python, data science, beginner” articles, represent those attributes as a taste profile and retrieve articles pointing in the same feature direction. Unlike collaborative filtering, it does not require other users to have consumed the candidate.

3. Prerequisites

Vector algebra, cosine similarity, TF-IDF, categorical encoding, feature scaling, basic classification/ranking, and sparse matrices.

4. Core Concepts

- **Item representation:** a vector built from metadata or learned encoders. Quality bounds the recommender’s quality. Example: TF-IDF of a movie synopsis. Interview: discuss sparse versus dense features.
- **User profile:** aggregate vectors of liked items, often weighted by ratings and recency. Example: mean of saved-article vectors. Interview: negative feedback should subtract or train a classifier.
- **Similarity/relevance:** cosine similarity or a learned score compares user and item. Interview: normalization prevents vector magnitude from dominating.
- **Feature engineering:** combine text, category, price, and brand without letting a high-dimensional block overwhelm others.

5. Algorithm / Working Process

Input item metadata and a user’s history. Encode each item, construct a user vector from positive/negative history, compute scores against eligible candidates, remove consumed items, and return top K . Classical versions need no separate training beyond fitting TF-IDF. Learned versions train an encoder or relevance model and cache item vectors for retrieval.

6. Mathematical Foundation

With item vector x_i and preference weights a_{ui} , a normalized profile is

$$p_u = (\sum_{i \in H_u} a_{ui} x_i) / (\sum_i |a_{ui}|).$$

Cosine relevance is

$$s(u,i) = (p_u^T x_i) / (\|p_u\|_2 \|x_i\|_2).$$

TF-IDF uses $\text{tfidf}(t,d) = \text{tf}(t,d) \log(N/\text{df}(t))$. A supervised variant learns $P(y_{ui}=1) = \text{sigmoid}(w^T \phi(p_u, x_i))$ with binary cross-entropy $-y \log p - (1-y) \log(1-p)$.

7. Practical Implementation

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize

items = pd.DataFrame({
    "item_id": [10, 11, 12, 13],
    "text": ["python machine learning", "deep learning pytorch",
            "italian cooking pasta", "python data analysis pandas"]
})
tfidf = TfidfVectorizer(stop_words="english")
X = tfidf.fit_transform(items["text"])

liked_ids = [10, 11]
liked_rows = items.index[items.item_id.isin(liked_ids)]
profile = normalize(X[liked_rows].mean(axis=0))
scores = np.asarray(X @ profile.T).ravel() # X is already L2-normalized
scores[liked_rows] = -np.inf
top = np.argsort(-scores)[:2]
print(items.loc[top, ["item_id", "text"]].assign(score=scores[top]))
```

8. Code Explanation

TF-IDF converts descriptions to normalized sparse vectors. Averaging liked rows creates the user profile; the sparse matrix product computes cosine scores for all items at once. Consumed items are masked before selecting the highest scores. Real systems fit vocabulary on training data and combine multiple feature blocks.

9. Training / Evaluation

Split interactions chronologically, but fit text preprocessing without test-derived behavioral labels. Sample negatives carefully and evaluate Recall@K/NDCG@K plus catalog coverage and intra-list diversity. Tune n-grams, minimum document frequency, feature-block weights, profile history length, and recency decay. Inspect recommendations qualitatively for overspecialization.

10. Complexity and Cost

TF-IDF fitting is roughly linear in corpus tokens. Brute-force scoring is $O(I \cdot d)$ per user, reduced by sparse operations or approximate nearest-neighbor (ANN) indexes. Item storage is $O(I \cdot d)$. Classical models run on CPU; transformer encoders often require GPU training but allow cached item embeddings.

11. Common Use Cases

Similar articles, related products, job-candidate matching, music by audio attributes, academic-paper discovery, and cold-start item retrieval.

12. Common Mistakes

- Using only positive history and never modeling dislikes
- Mixing unscaled feature blocks
- Fitting vocabularies or encoders using test labels
- Recommending already consumed/unavailable items
- Treating cosine similarity as calibrated click probability
- Ignoring vocabulary, language, and metadata quality

13. Edge Cases / Limitations

It overspecializes, struggles with a new user, cannot discover taste signals absent from metadata, and may recommend near duplicates. Poor descriptions create poor vectors. A user with multiple interests may be blurred into one average profile.

14. Variations

- **Rocchio profile:** positive centroid minus negative centroid; useful with explicit feedback; placement-relevant.
- **Supervised content ranker:** learns feature weights from clicks; use at scale; project-important.
- **Multimodal encoders:** combine text/image/audio; research-relevant.
- **Multiple-interest profiles:** cluster history into several tastes; useful for diverse users.

15. Related Topics

Embeddings replace manual features with learned semantics. Two-tower models learn compatible user/item vectors, while hybrid recommenders combine content with collaborative signals to address both item cold start and behavioral nuance.

16. Interview Questions

1. **What signal drives content filtering?** Similarity between item attributes and a user profile.
2. **How is a user profile built?** Weighted aggregation or a learned model over historical item features.
3. **Why cosine similarity?** It compares direction and reduces magnitude effects.
4. **Does it solve cold start?** New-item cold start, if metadata exists; not new-user cold start by itself.
5. **What is overspecialization?** Recommending only items very similar to past choices.
6. **How do dislikes enter?** Subtract negative centroids or learn from labeled positive/negative examples.
7. **Why normalize feature blocks?** Otherwise high-dimensional or large-scale features dominate.
8. **TF-IDF versus embeddings?** TF-IDF is lexical and cheap; embeddings capture semantics but cost more.
9. **How do you support multiple tastes?** Maintain multiple clustered profiles or attend over history.

10. **Main production optimization?** Precompute item vectors and use ANN retrieval.

17. Practice Tasks

Build a TF-IDF movie recommender; add genre one-hot features; compare cosine with a logistic ranker; diagnose near-duplicate outputs; extend the profile with recency and negative feedback.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Job matcher	Matches resumes to vacancies	sklearn/spaCy; Kaggle jobs	Explainable semantic matching
Paper finder	Retrieves related abstracts	Sentence Transformers/FAISS; arXiv	Embeddings and ANN retrieval
Fashion similarity	Uses image and metadata	PyTorch/CLIP; DeepFashion	Multimodal recommendation

19. Quick Revision

- **Key idea:** recommend feature-similar items.
- **Formula:** cosine between user profile and item vector.
- **Use:** rich metadata and new items.
- **Metrics:** Recall@K, NDCG@K, diversity.
- **Trap:** overspecialization and bad feature scaling.
- **One-liner:** content models know what an item is, not who else liked it.

20. Final Cheat Sheet

Item	Summary
Input/output	Item features + history -> ranked items
Steps	Encode, profile, score, filter, top-K
Hyperparameters	Vectorizer/encoder, feature weights, history decay
Pros	Explainable, handles new items, independent of crowd
Cons	Metadata-bound, overspecialized, weak new-user support
Best use	News, jobs, documents, cold catalog items

Collaborative Filtering

1. Overview

Collaborative filtering (CF) predicts preferences from patterns in user-item interactions. It assumes users who behaved similarly will prefer similar items, even when item metadata is unavailable. CF powers media, retail,

social, and marketplace personalization. Neighborhood methods and matrix factorization are its classical forms.

2. Intuition

Two viewers who independently liked many of the same films provide evidence about each other's unseen choices. CF uses the crowd as a distributed labeling system: an item's meaning is inferred from who interacted with it.

3. Prerequisites

Sparse matrices, vector similarity, train/test splitting, explicit versus implicit feedback, linear algebra, regularization, and ranking metrics.

4. Core Concepts

- **Interaction matrix:** rows are users, columns items, and most entries are missing. Missing usually means unknown, not dislike.
- **Explicit feedback:** ratings carry preference strength but are scarce and selection-biased.
- **Implicit feedback:** clicks, watches, and purchases are abundant but noisy; confidence differs from preference.
- **Neighborhood vs model-based CF:** nearest neighbors use local overlap; factor models learn global latent structure.
- **Biases:** user generosity and item popularity should be separated from personalized affinity.

5. Algorithm / Working Process

Map IDs to indices, build interactions, choose an explicit or implicit objective, split by time, fit neighbor statistics or latent parameters, score unseen eligible items, and return top K . During inference, retrieve candidates from similar users/items or item latent vectors, then filter seen and invalid items.

6. Mathematical Foundation

A baseline rating model is $\hat{r}_{ui} = \mu + b_u + b_i$. Neighborhood CF adds a weighted residual from neighbors. Latent CF uses $\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$. Explicit models minimize squared error over observed set Ω :

$$L = \sum_{(u,i) \in \Omega} (r_{ui} - \hat{r}_{ui})^2 + \lambda(|p_u|^2 + |q_i|^2 + b_u^2 + b_i^2).$$

For implicit feedback, pairwise BPR minimizes $-\log \sigma(s_{ui} - s_{uj})$ for positive item i and sampled negative j .

7. Practical Implementation

```
import numpy as np

R = np.array([[5., 4., 0., 1.],
              [4., 0., 0., 1.],
              [1., 1., 0., 5.]])
```

```

        [0., 1., 5., 4.]))
mask = R > 0
k, lr, reg = 2, 0.01, 0.05
rng = np.random.default_rng(42)
P = rng.normal(0, .1, (R.shape[0], k))
Q = rng.normal(0, .1, (R.shape[1], k))

for _ in range(1000):
    for u, i in np.argwhere(mask):
        err = R[u, i] - P[u] @ Q[i]
        pu = P[u].copy()
        P[u] += lr * (err * Q[i] - reg * P[u])
        Q[i] += lr * (err * pu - reg * Q[i])

scores = P @ Q.T
scores[mask] = -np.inf
print("recommendations for user 0:", np.argsort(-scores[0])[:2])

```

8. Code Explanation

Only observed cells generate squared-error updates. Each user and item receives a two-dimensional latent vector; their dot product predicts affinity. Regularization shrinks vectors to reduce overfitting. The mask is critical: zeros are missing values here, and consumed items are excluded at serving.

9. Training / Evaluation

Use time-based splits and ensure every validation/test user is representable, or report cold users separately. For implicit data, create negatives from eligible but unobserved items and avoid sampling impossible exposures. Evaluate Recall@K, NDCG@K, MAP, coverage, novelty, calibration, and online CTR/conversion. Tune dimension, regularization, learning rate, epochs, negative count, and confidence weights.

10. Complexity and Cost

Neighborhood similarity can cost $O(U^2 I)$ or $O(I^2 U)$ naively. Factorization SGD costs $O(Ek)$ per epoch and stores $O((U+I)k)$. Large embedding tables consume substantial memory; distributed CPU/GPU training and ANN retrieval are common.

11. Common Use Cases

Movie, music, product, creator, course, ad, and feed recommendation where repeated users and items generate interaction history.

12. Common Mistakes

- Treating every missing interaction as a true negative
- Randomly splitting events and leaking future behavior
- Allowing duplicates or seen items in top-K
- Evaluating rating RMSE when the product needs ranking
- Ignoring exposure/position bias and bot events
- Forgetting cold users/items cannot have learned factors

13. Edge Cases / Limitations

CF fails for cold entities, suffers under extreme sparsity, and amplifies popularity/exposure biases. Preferences drift, latent factors may be hard to explain, and feedback loops make logged data non-i.i.d. Shilling attacks can manipulate recommendations.

14. Variations

- **User-user/item-item CF:** interpretable local neighborhoods; placement-essential.
- **SVD/ALS/BPR:** scalable latent models for explicit or implicit data; project-essential.
- **Hybrid CF:** incorporates metadata for cold start; production-important.
- **Context-aware/sequential CF:** adds time and order; research and feed applications.

15. Related Topics

User-user and item-item similarity are memory-based CF. Matrix factorization compresses interactions into embeddings. Neural CF adds nonlinear interaction functions, while graph recommenders propagate collaborative signals over the bipartite graph.

16. Interview Questions

1. **What does collaborative mean?** Preferences are inferred from behavior shared across users.
2. **Explicit vs implicit feedback?** Ratings express preference; events indicate noisy, exposure-dependent behavior.
3. **Is missing equal to negative?** Usually no; the user may never have seen the item.
4. **Memory-based vs model-based?** Neighbor lookup versus learned parameters.
5. **Why is sparsity difficult?** Similarity and latent parameters have little evidence.
6. **Why include bias terms?** They capture user scale and global item popularity separately.
7. **Best split?** Chronological per-user or global temporal split matching deployment.
8. **Why can RMSE be misleading?** Accurate rating values do not guarantee good top-K order.
9. **How do you scale retrieval?** Precompute factors and use ANN maximum-inner-product search.
10. **How do you handle cold start?** Metadata, popularity, onboarding, or hybrid models.

17. Practice Tasks

Implement neighbor CF and factorization; compare explicit RMSE with NDCG@10; vary sparsity; debug a split with future leakage; add bias terms and implicit negative sampling.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Movie recommender	Compares KNN, SVD, BPR	NumPy/PyTorch; MovieLens	Full offline evaluation
Retail cross-sell	Learns from purchases	implicit/FAISS; Retailrocket	Sparse implicit feedback
Course discovery	Hybrid CF for learners	FastAPI/PyTorch; OULAD	Cold-start and deployment

19. Quick Revision

- **Key idea:** learn taste from interaction patterns.
- **Formula:** bias plus user-item latent dot product.
- **Use:** recurring users/items with behavioral history.
- **Metrics:** Recall/NDCG/MAP@K; RMSE for rating prediction.
- **Trap:** missing is not negative.
- **One-liner:** CF learns item meaning from the company it keeps.

20. Final Cheat Sheet

Item	Summary
Input/output	Sparse interactions -> preference scores/top-K
Steps	Split, encode, fit neighbors/factors, rank, filter
Hyperparameters	Neighbors/dimension, regularization, negatives
Pros	No metadata required; captures subtle taste
Cons	Cold start, sparsity, bias, feedback loops
Best use	Mature catalogs with repeated interaction

User-User Similarity

1. Overview

User-user collaborative filtering recommends items liked by users whose historical preferences resemble the target user. It is interpretable and useful for small or moderately sized communities, educational examples, and domains where user neighborhoods are meaningful.

2. Intuition

Ask people with taste similar to yours what they liked that you have not tried. Agreement on several uncommon items is stronger evidence than agreement only on blockbusters.

3. Prerequisites

Sparse user-item matrices, cosine/Pearson similarity, mean centering, nearest-neighbor search, and top-K evaluation.

4. Core Concepts

- **Co-rated overlap:** similarity must use items both users rated; tiny overlap is unreliable.
- **Cosine similarity:** compares interaction-vector direction; suitable for implicit or centered explicit data.
- **Pearson correlation:** compares deviations from each user's mean; corrects different rating scales.
- **Neighborhood:** top similar users, often after minimum-overlap and positive-similarity filters.
- **Significance weighting:** shrink similarities computed from few shared items.

5. Algorithm / Working Process

Build a user-item matrix, normalize rows, find users sharing items with target u , compute similarity, retain top N , and score each unseen item by neighbors' weighted preferences. Return eligible top K . There is no learned parametric model; offline computation can cache neighbors.

6. Mathematical Foundation

Cosine similarity is $\text{sim}(u,v) = \frac{r_u^T r_v}{\|r_u\| \|r_v\|}$. Pearson on co-rated set I_{uv} is

$$\text{sim}(u,v) = \frac{\sum_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_i (r_{ui} - \bar{r}_u)^2 \sum_i (r_{vi} - \bar{r}_v)^2)}$$

Prediction is

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_u(i)} \text{sim}(u,v)(r_{vi} - \bar{r}_v)}{\sum_v |\text{sim}(u,v)|}$$

Shrink similarity with $\text{sim}' = \text{sim} * \min(|I_{uv}|/\beta, 1)$.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

R = np.array([[5., 4., 0., 1.], [4., 5., 0., 1.],
              [1., 0., 5., 4.], [0., 1., 4., 5.]])
observed = R > 0
means = np.divide(R.sum(1), observed.sum(1), where=observed.sum(1) != 0)
centered = np.where(observed, R - means[:, None], 0)
S = cosine_similarity(centered)
np.fill_diagonal(S, 0)

u = 0
numerator = S[u] @ centered
denominator = np.abs(S[u]) @ observed
pred = means[u] + np.divide(numerator, denominator,
                           out=np.zeros_like(numerator), where=denominator > 0)
pred[observed[u]] = -np.inf
print(np.argsort(-pred)[:2], pred[np.argsort(-pred)[:2]])
```

8. Code Explanation

Mean centering turns ratings into above/below-normal preferences. Cosine over centered rows approximates Pearson behavior. Matrix multiplication accumulates similarity-weighted residuals, and the denominator normalizes only by neighbors who rated that item. Zero support must trigger a fallback rather than an invented score.

9. Training / Evaluation

Tune neighbor count, overlap threshold, shrinkage, and similarity on a temporal validation split. Evaluate top-K ranking and, for explicit ratings, MAE/RMSE. Report coverage: strict overlap filters can leave many users without neighbors. Compare against item-item and popularity baselines.

10. Complexity and Cost

Naive all-pairs similarity is $O(U^2 I)$ time and $O(U^2)$ storage. Inverted indexes restrict comparisons to co-interacting users, and ANN can retrieve neighbors. Online scoring scales with neighborhood interactions. Rapid user growth makes this less stable than item-item CF.

11. Common Use Cases

Book clubs, course communities, social discovery, small streaming catalogs, and explainable prototypes such as “users similar to you liked...”.

12. Common Mistakes

Computing similarity over zeros as real ratings; skipping mean centering for explicit ratings; trusting one shared item; including the target user as its own neighbor; dividing by signed similarity sum; leaking held-out events.

13. Edge Cases / Limitations

New or low-activity users have no reliable neighbors. Large user populations make neighbor indexes expensive and volatile. Users with mainstream behavior may look spuriously similar; malicious profiles can influence neighborhoods.

14. Variations

- **Pearson CF:** corrects rating-scale differences; placement-essential.
- **Jaccard CF:** compares binary sets; useful for implicit events.
- **Shrunk cosine:** discounts small overlap; production-important.
- **Demographic neighborhoods:** helps cold start but raises fairness/privacy concerns.

15. Related Topics

Item-item CF transposes the neighborhood direction and is often more stable. Matrix factorization replaces explicit neighbors with latent vectors. Jaccard, cosine, and Pearson are also used in clustering and information retrieval.

16. Interview Questions

1. **How are neighbors chosen?** Highest reliable similarity among users with overlap.
2. **Cosine vs Pearson?** Pearson centers individual rating scales; raw cosine does not.
3. **Why minimum overlap?** Similarity from one coincidence has high variance.
4. **Why absolute denominator?** Negative weights should not cancel normalization mass.
5. **What if no neighbor rated an item?** Use item/popularity/global fallback.
6. **Can it use implicit data?** Yes, with cosine/Jaccard and weighted events.
7. **How do you explain a result?** Cite similar users and contributing liked items, respecting privacy.

8. **Why is it hard at scale?** User count and profiles change rapidly.
9. **What is mean centering?** Subtract each user's average rating before similarity/scoring.
10. **How do you prevent leakage?** Compute all profiles and neighbors using past-only events.

17. Practice Tasks

Implement Pearson from scratch; add overlap shrinkage; compare cosine and Jaccard; investigate users with zero recommendation coverage; create an explanation from top contributing neighbors.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Taste-neighbor explorer	Visualizes user neighborhoods	Streamlit/sklearn; MovieLens	Interpretability
Study buddy recommender	Connects learners by courses	Pandas/FastAPI; OULAD	Responsible similarity
Book club engine	Recommends from reader peers	scipy; Book-Crossing	Sparse KNN optimization

19. Quick Revision

- **Key idea:** borrow unseen preferences from similar users.
- **Formula:** weighted neighbor residual around user mean.
- **Use:** smaller, stable communities.
- **Metrics:** Recall/NDCG@K, coverage, RMSE.
- **Trap:** unreliable similarity from tiny overlap.
- **One-liner:** user-user CF finds taste peers, then aggregates their deviations.

20. Final Cheat Sheet

Item	Summary
Input/output	User-item matrix -> neighbor-weighted scores
Steps	Center, compare users, select neighbors, aggregate
Hyperparameters	Similarity, neighbor count, overlap, shrinkage
Pros	Intuitive and explainable
Cons	Sparse, volatile, expensive with many users
Best use	Moderate communities with repeated ratings

Item-Item Similarity

1. Overview

Item-item collaborative filtering recommends items behaviorally similar to those a user consumed. Two items are similar when many of the same users interact with or rate them similarly. It is widely used for “customers who bought this also bought” and item-to-item candidate retrieval because item relationships are often more stable than user relationships.

2. Intuition

Instead of finding people like a shopper, find products that attract the same shoppers. A user who bought a camera can receive lenses and memory cards because those item columns overlap in purchase histories.

3. Prerequisites

Sparse matrices, cosine/adjusted-cosine/Jaccard similarity, co-occurrence counts, nearest-neighbor retrieval, and implicit feedback.

4. Core Concepts

- **Co-occurrence:** count shared users; fast but dominated by popular items.
- **Item vector:** each item is represented by its interactions across users.
- **Adjusted cosine:** subtract user means before comparing item columns, controlling rating-scale differences.
- **Similarity shrinkage:** discounts pairs with little support.
- **Basket/session context:** item relationships may come from co-purchase or co-view windows rather than lifetime users.

5. Algorithm / Working Process

Build item interaction vectors, normalize or center them, compute supported item pairs, retain each item’s top neighbors, and cache the graph. At request time, gather neighbors of recent/weighted history items, sum contributions, filter invalid/seen items, and select top K .

6. Mathematical Foundation

Binary cosine is $\text{sim}(i,j) = |U_i \cap U_j| / \sqrt{|U_i| |U_j|}$. Jaccard is $|U_i \cap U_j| / |U_i \cup U_j|$. Adjusted cosine is

$$\text{sim}(i,j) = \frac{\sum_u (r_{ui} - \bar{r}_u)(r_{uj} - \bar{r}_u)}{\sqrt{\sum_u (r_{ui} - \bar{r}_u)^2 \sum_u (r_{uj} - \bar{r}_u)^2}}$$

A user score is $s(u,j) = \sum_{i \in H_u} w_{ui} \text{sim}(i,j)$, optionally normalized. Log-likelihood ratio or pointwise mutual information can reduce raw popularity effects.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# rows users, columns items; binary implicit feedback
X = np.array([[1, 1, 0, 0], [1, 1, 1, 0],
```

```

        [0, 1, 1, 1], [0, 0, 1, 1]], dtype=float)
item_sim = cosine_similarity(X.T)
np.fill_diagonal(item_sim, 0)

u = 0
scores = X[u] @ item_sim          # sum neighbors of user's items
scores[X[u] > 0] = -np.inf        # do not repeat consumed items
top = np.argsort(-scores)[:2]
print("top items:", top, "scores:", scores[top])

```

8. Code Explanation

Transposing makes items the samples and users their features. Cosine normalizes support, so a universally popular item does not win solely by count. The user's history weights rows of the item-similarity matrix. Production code stores only top neighbors and may emphasize recent or high-value seed items.

9. Training / Evaluation

Construct item pairs using training-period interactions only. Tune neighbor count, support threshold, shrinkage, history length, and event weights. Evaluate hit rate/Recall/NDCG@K, coverage, novelty, and diversity on future events. For cart recommendation, evaluate within-basket next-item prediction rather than arbitrary lifetime splits.

10. Complexity and Cost

Naive similarity is $O(I^2 U)$ and $O(I^2)$ memory. Sparse co-occurrence enumerates pairs within each user/session and stores only L neighbors per item: $O(IL)$ memory. Online retrieval is about $O(|H_u|L)$ before top-K selection. CPU sparse jobs are usually adequate.

11. Common Use Cases

Related products, frequently bought together, similar videos, next article, playlist continuation, substitution and complement discovery, and candidate retrieval.

12. Common Mistakes

Keeping self-similarity; using raw co-counts without popularity normalization; computing pairs across future events; failing to distinguish substitutes from complements; letting very long user histories create quadratic pair noise; not filtering consumed/out-of-stock items.

13. Edge Cases / Limitations

New items lack interaction neighbors. Rare pairs are unstable, popular items dominate, and similarity does not reveal whether items substitute or complement one another. Static neighbors lag fast trends and ignore the order of a session.

14. Variations

- **Adjusted cosine:** explicit-rating data; placement-essential.

- **Jaccard/association rules:** baskets and binary events; project-important.
- **Time/windowed co-occurrence:** sessions and news; production-important.
- **Learned item embeddings:** replace hand-designed similarity; important for deep learning roles.

15. Related Topics

User-user CF finds peer rows; item-item CF compares columns. Association rules add directional confidence/lift. Embeddings and graph methods generalize item proximity beyond direct co-occurrence.

16. Interview Questions

1. **Why can item-item scale better?** Catalogs and item relations often change more slowly than user profiles.
2. **Cosine vs Jaccard?** Cosine normalizes geometrically; Jaccard normalizes by union size.
3. **What is adjusted cosine?** Item cosine after subtracting each user's rating mean.
4. **How are candidates scored?** Aggregate similarity from the user's seed/history items.
5. **How do you handle popularity bias?** Normalize, shrink, use lift/PMI, or re-rank.
6. **Similarity or complementarity?** Co-purchase may find complements; metadata similarity often finds substitutes.
7. **How do new items work?** Content-based neighbors until behavioral evidence arrives.
8. **Why top-L storage?** Full $I \times I$ similarity is too large and mostly irrelevant.
9. **How do sessions help?** They create intent-specific, temporally local co-occurrences.
10. **Main leakage risk?** Building similarities from interactions after the evaluation cutoff.

17. Practice Tasks

Implement cosine/Jaccard; add minimum support and shrinkage; compare lifetime versus session co-occurrence; inspect popularity bias; extend scoring with recency-weighted history.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Cart companion	Suggests complementary items	scipy/FastAPI; Instacart	Basket-aware retrieval
Video neighbors	Builds related-video graph	Spark/ANN; MovieLens	Scalable sparse similarity
Playlist continuer	Scores next tracks by co-listen	Python; Last.fm	Temporal weighting and diversity

19. Quick Revision

- **Key idea:** aggregate neighbors of items in user history.
- **Formula:** normalized item-column overlap.
- **Use:** stable catalogs and related-item retrieval.
- **Metrics:** Recall/NDCG@K, coverage, diversity.
- **Trap:** raw co-count popularity and temporal leakage.

- **One-liner:** item-item CF turns collective co-behavior into a reusable neighbor graph.

20. Final Cheat Sheet

Item	Summary
Input/output	Interactions + seed items -> related/top-K items
Steps	Co-occur, normalize, retain neighbors, aggregate
Hyperparameters	Similarity, support, shrinkage, neighbor count
Pros	Stable, interpretable, fast online
Cons	Cold items, pair-computation cost, popularity bias
Best use	Product/video related-item candidates

Matrix Factorization

1. Overview

Matrix factorization (MF) approximates a sparse user-item interaction matrix with low-dimensional user and item factors. It is a central model for explicit ratings and implicit-feedback recommendation because it captures latent tastes without item metadata. SVD-style models, alternating least squares (ALS), weighted MF, and Bayesian personalized ranking (BPR) are common variants.

2. Intuition

A movie may have hidden coordinates such as “serious versus light” and “action versus romance”; a user has preference coordinates on the same axes. Their dot product measures compatibility. The dimensions are learned from behavior rather than named in advance.

3. Prerequisites

Matrix multiplication, dot products, low-rank approximation, gradient descent/ALS, regularization, sparse data, explicit versus implicit objectives, and negative sampling.

4. Core Concepts

- **Latent factors:** vectors p_u, q_i in $\mathbb{R}^k$; they compress interaction patterns. Interview angle: factors need not be human-interpretable.
- **Bias terms:** global, user, and item offsets capture easy systematic effects before latent interaction.
- **Observed-only loss:** explicit MF trains only where ratings exist; zero is usually not a rating.
- **Implicit confidence:** an event creates preference p_{ui} and confidence c_{ui} ; unobserved entries receive low, nonzero confidence.
- **Regularization:** controls vector magnitude and prevents rare entities from memorizing noise.

5. Algorithm / Working Process

Map user/item IDs, initialize factors and biases, predict by dot product, measure an appropriate loss, and update parameters with SGD or alternating least squares. Training repeats over observations or sampled triplets. Inference obtains a user vector, computes inner products with eligible item vectors, uses ANN for a large catalog, filters, and returns top K .

6. Mathematical Foundation

Prediction:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i.$$

Explicit objective:

$$\min \sum_{(u,i) \in \Omega} (r_{ui} - \hat{r}_{ui})^2 + \lambda (||P||_F^2 + ||Q||_F^2 + ||b||_2^2).$$

For weighted implicit ALS, define $p_{ui}=1[r_{ui}>0]$, $c_{ui}=1+\alpha r_{ui}$ and minimize

$$\sum_{(u,i)} c_{ui} (p_{ui} - p_u^T q_i)^2 + \lambda (||P||_F^2 + ||Q||_F^2).$$

BPR samples (u,i,j) and minimizes $-\sum \log \sigma(p_u^T q_i - p_u^T q_j) + \lambda ||\Theta||^2$, directly encouraging positives above negatives. ALS alternately solves regularized least-squares systems for all user factors while holding items fixed, then vice versa.

7. Practical Implementation

```
import torch
from torch import nn

class MatrixFactorization(nn.Module):
    def __init__(self, n_users, n_items, dim=32):
        super().__init__()
        self.user = nn.Embedding(n_users, dim)
        self.item = nn.Embedding(n_items, dim)
        self.user_bias = nn.Embedding(n_users, 1)
        self.item_bias = nn.Embedding(n_items, 1)

    def forward(self, users, items):
        dot = (self.user(users) * self.item(items)).sum(dim=1)
        return dot + self.user_bias(users).squeeze(1) + self.item_bias(items).squeeze(1)

# Explicit-rating training step
model = MatrixFactorization(n_users=1000, n_items=2000)
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
users = torch.tensor([0, 1, 1, 2])
items = torch.tensor([3, 8, 9, 3])
ratings = torch.tensor([5., 4., 1., 3.])
loss = nn.functional.mse_loss(model(users, items), ratings)
opt.zero_grad(); loss.backward(); opt.step()
print(float(loss))
```

8. Code Explanation

Embedding tables are simply factor matrices addressed by integer IDs. Elementwise multiplication followed by a sum is a batched dot product. Bias embeddings learn per-entity offsets. `weight_decay` supplies L2 regularization. For implicit ranking, replace MSE with BCE over sampled negatives or the BPR loss over triplets.

9. Training / Evaluation

Use a chronological split and fit ID mappings on training data with unknown fallbacks. Explicit rating tasks use RMSE/MAE; top-K products use Recall, NDCG, MAP, or MRR. Tune `k`, regularization, learning rate, epochs, confidence `alpha`, and negative sampling. Monitor train-validation gaps; larger dimensions especially overfit rare entities.

10. Complexity and Cost

SGD costs $O(Ek)$ per epoch and memory is $O((U+I)k)$. Implicit ALS avoids enumerating a dense matrix through sparse algebra but solves $k \times k$ systems. Full scoring costs $O(Ik)$ per user; ANN reduces retrieval latency. Large ID tables may require sharding, mixed precision, and CPU-hosted embeddings.

11. Common Use Cases

Movie ratings, music listening, product purchases, ad response, course enrollment, and a retrieval baseline for any mature interaction graph.

12. Common Mistakes

Applying ordinary dense SVD after filling missing ratings with zero; using MSE for a top-K goal without comparison; sampling clicked items as negatives; evaluating known test users with factors trained on their future interactions; forgetting biases or seen-item filtering.

13. Edge Cases / Limitations

Unseen IDs have no factors, dot products express only limited interaction structure, and static factors lag preference drift. Rare entities are poorly estimated. Exposure bias remains because training data records recommendations made by earlier policies.

14. Variations

- **Biased SVD/Funk SVD:** explicit ratings; placement-essential.
- **SVD++:** adds implicit-history factors; important advanced interview topic.
- **Implicit ALS:** confidence-weighted binary data; production/project-essential.
- **BPR-MF:** pairwise top-K optimization; placement and research relevant.
- **Factorization machines:** add sparse contextual features; useful for hybrid systems.

15. Related Topics

Embeddings are the learned rows of factor matrices. Neural CF replaces or augments the dot product with a network. Two-tower retrieval generalizes factorization by producing factors from user/item features, solving some cold-start cases.

16. Interview Questions

1. **Why low rank?** A few latent tastes explain much of the interaction structure.
2. **Why not fill missing ratings with zero?** Missing usually means unobserved, not dislike.
3. **What do bias terms capture?** Global mean, user scale, and item popularity/quality.
4. **SGD vs ALS?** SGD is flexible; ALS alternates convex subproblems and parallelizes well.
5. **What is implicit confidence?** More/repeated interaction raises confidence in binary preference.
6. **What does BPR optimize?** The positive-negative score ordering for a user.
7. **How is k selected?** Validation ranking/error versus latency and memory.
8. **How are recommendations served?** Maximum-inner-product retrieval over item factors.
9. **Can factors be interpreted?** Sometimes post hoc, but dimensions are rotationally non-unique.
10. **How do you handle unseen IDs?** Feature-based/hybrid towers or fallback models.

17. Practice Tasks

Add biases to MF; implement BPR triplet loss; compare MSE and NDCG; graph validation score versus dimension; debug negative sampling that includes positives.

18. Project Ideas

Project	What it does	Stack and data	Resume value
MF laboratory	Compares SGD, ALS, BPR	PyTorch/implicit; MovieLens	Objective and metric reasoning
Retail retrieval	Serves factor ANN candidates	PyTorch/FAISS; Retailrocket	Training-to-serving pipeline
Explainable factors	Visualizes latent item space	Plotly/sklearn; Last.fm	Representation analysis

19. Quick Revision

- **Key idea:** approximate interactions by user/item latent dot products.
- **Formula:** $\mu + b_u + b_i + p_u^T q_i$.
- **Use:** sparse recurring interactions.
- **Metrics:** RMSE for ratings; NDCG/Recall for ranking.
- **Trap:** treating unknown as dislike.
- **One-liner:** MF compresses collaborative behavior into compatible latent factors.

20. Final Cheat Sheet

Item	Summary
Input/output	Sparse interactions -> factors and scores
Steps	Initialize, predict, optimize, ANN retrieve
Hyperparameters	Dimension, regularization, objective, negatives
Pros	Strong, scalable, compact

Item	Summary
Cons	Cold start, linear interaction, exposure bias
Best use	Collaborative retrieval/rating prediction

Embeddings

1. Overview

An embedding is a learned dense vector representing a user, item, category, query, or context so that geometric relationships encode useful semantics. In recommendation, embeddings support retrieval, ranking features, clustering, visualization, transfer, and similarity search. MF factors, Word2Vec-style item vectors, graph embeddings, and neural tower outputs are all embeddings.

2. Intuition

Give every entity coordinates on a learned taste map. Items frequently consumed in similar contexts become neighbors, and a user vector points toward appealing regions. The map is useful because distance or dot product replaces expensive reasoning over raw histories.

3. Prerequisites

Vectors, dot products and norms, neural embeddings, categorical IDs, sampling, softmax/BCE losses, sequence windows, and ANN indexes.

4. Core Concepts

- **ID embeddings:** lookup-table rows; expressive but unavailable for new IDs.
- **Feature embeddings:** produced from text/images/categories; generalize to new entities.
- **Metric choice:** dot product includes vector magnitude; cosine uses direction; Euclidean distance defines local geometry.
- **Positive pairs and negatives:** the training construction determines what “near” means.
- **Embedding collapse/anisotropy:** vectors may become indistinguishable or occupy a narrow cone; monitor norm and neighbor distributions.

5. Algorithm / Working Process

Choose entities and a notion of relatedness, encode IDs/features, create positive pairs from interactions or sequence windows, sample negatives, optimize a similarity objective, export vectors, build an ANN index, and retrieve neighbors. Rebuild or incrementally update vectors as catalog and behavior change.

6. Mathematical Foundation

Similarity may be $s(a,b)=e_a^T e_b/\tau$ or cosine divided by temperature τ . Skip-gram negative sampling maximizes

$$\log \sigma(e_i^T e_j) + \sum_{n \in N} \log \sigma(-e_i^T e_n).$$

InfoNCE for one positive $i+$ among batch candidates is

$$L = -\log[\exp(s(u, i+)/\tau) / \sum_j \exp(s(u, j)/\tau)].$$

Gradients pull positives together and push negatives apart. L2 normalization makes dot product equal cosine and prevents norm from acting as unbounded popularity.

7. Practical Implementation

```
import torch
from torch import nn

item_embedding = nn.Embedding(5000, 64)
optimizer = torch.optim.Adam(item_embedding.parameters(), 1e-3)

# center item and true context item from user sequences
center = torch.tensor([10, 25, 80, 91])
positive = torch.tensor([12, 29, 81, 95])
negative = torch.randint(0, 5000, (len(center), 20))

anchor = item_embedding(center)
pos_score = (anchor * item_embedding(positive)).sum(-1)
neg_score = torch.einsum("bd,bnd->bn", anchor, item_embedding(negative))
loss = -(torch.nn.functional.logsigmoid(pos_score).mean()
          + torch.nn.functional.logsigmoid(-neg_score).mean())
optimizer.zero_grad(); loss.backward(); optimizer.step()
print(float(loss))
```

8. Code Explanation

The lookup table maps item IDs to 64-dimensional vectors. Positive pairs come from nearby sequence items; random negatives approximate the full catalog. The loss raises positive dot products and lowers negative ones. Production sampling should exclude true positives and often draw harder or popularity-adjusted negatives.

9. Training / Evaluation

Split events by time before creating windows. Evaluate the downstream task: Recall@K for retrieval, NDCG for recommendation, plus neighbor sanity checks, coverage, norm distribution, and subgroup performance. Tune dimension, context window, negative count/distribution, temperature, normalization, and refresh frequency.

10. Complexity and Cost

Training is roughly $O(E \cdot d \cdot (1+n_{\text{neg}}))$; table memory is $O(Nd)$. Vector retrieval with brute force is $O(Nd)$, while ANN trades some recall for sublinear practical latency. GPUs help large contrastive batches; compressed or quantized embeddings reduce serving memory.

11. Common Use Cases

Similar-item search, candidate retrieval, "complete the playlist," semantic product search, user clustering, ranker features, anomaly detection, and visualization.

12. Common Mistakes

Calling embeddings inherently semantic; choosing negatives that are actually positives; evaluating only a 2-D visualization; using inconsistent training/serving normalization; allowing stale ID maps; comparing Euclidean and dot-product indexes incorrectly.

13. Edge Cases / Limitations

Rare IDs are poorly learned, new IDs need features, geometry can encode societal/exposure bias, and one vector may blur multiple interests. Approximate indexes miss some neighbors; vector drift complicates cached indexes.

14. Variations

- **MF embeddings:** user/item lookup factors; placement-essential.
- **Item2Vec:** sequence co-occurrence with skip-gram; project-important.
- **Content/multimodal embeddings:** encode cold items; production/research important.
- **Graph embeddings:** incorporate multi-hop connectivity; research-important.
- **Multi-vector embeddings:** represent several user interests; advanced retrieval.

15. Related Topics

Matrix factorization learns embeddings by reconstruction; two towers learn them through feature encoders; graph recommenders smooth them over edges; ANN systems serve them; ranking models often consume them as features.

16. Interview Questions

1. **What is an embedding?** A learned dense vector whose geometry supports a task.
2. **Dot product vs cosine?** Dot product uses angle and norm; cosine removes norm.
3. **Why negative sampling?** Full-catalog normalization is expensive.
4. **What makes a hard negative?** A plausible but non-positive item the model currently confuses.
5. **Can one embedding serve every task?** Not necessarily; geometry follows its training objective.
6. **How do embeddings handle cold items?** Use a metadata/content encoder rather than ID lookup alone.
7. **What is temperature?** A scale controlling softmax sharpness and gradient concentration.
8. **How are vectors served?** Cache item vectors in an ANN index and query with user vectors.
9. **How do you detect collapse?** Inspect variance, norms, pairwise similarities, and retrieval diversity.
10. **Why might popular items have large norms?** Frequent updates and dot-product objectives can encode popularity in magnitude.

17. Practice Tasks

Train Item2Vec from sessions; compare cosine/dot neighbors; vary negative distributions; measure ANN recall-latency; debug accidental positive negatives and index normalization mismatch.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Item2Vec explorer	Learns product neighborhoods	PyTorch/FAISS; Instacart	Contrastive learning and ANN
Multimodal catalog	Searches text/image products	CLIP/Qdrant; DeepFashion	Cold-start representations
Embedding monitor	Detects drift and collapse	Evidently/Plotly; synthetic logs	Production ML diagnostics

19. Quick Revision

- **Key idea:** encode entities so useful relations become geometric.
- **Formula:** contrastive softmax or positive/negative logistic loss.
- **Use:** retrieval and compact ranker features.
- **Metrics:** Recall@K, ANN recall, coverage, latency.
- **Trap:** geometry is objective-dependent.
- **One-liner:** embeddings turn recommendation entities into searchable task-specific coordinates.

20. Final Cheat Sheet

Item	Summary
Input/output	IDs/features/pairs -> dense vectors
Steps	Pair, encode, contrast, index, retrieve
Hyperparameters	Dimension, negatives, temperature, normalization
Pros	Compact, reusable, fast retrieval
Cons	Cold IDs, bias, drift, limited interpretability
Best use	Similarity search and candidate generation

Neural Collaborative Filtering

1. Overview

Neural collaborative filtering (NCF) replaces the fixed user-item dot product with a neural interaction function. The canonical design combines generalized matrix factorization (GMF) and a multilayer perceptron (MLP), called NeuMF. It is useful when nonlinear interactions improve ranking, though a well-tuned MF baseline is often competitive and cheaper.

2. Intuition

MF says every latent dimension combines only through matching coordinates and addition. NCF lets a network learn rules such as “this user’s taste for niche science fiction matters only when the item is recent and serialized,” provided features and data expose that pattern.

3. Prerequisites

Collaborative filtering, embeddings, MLPs, BCE, negative sampling, backpropagation, regularization, and top-K evaluation.

4. Core Concepts

- **GMF branch:** elementwise user-item product generalizes MF.
- **MLP branch:** concatenated embeddings pass through nonlinear layers.
- **NeuMF fusion:** concatenates GMF and MLP representations before the output.
- **Pointwise implicit objective:** classify observed versus sampled unobserved pairs.
- **Negative sampling:** crucial for compute and the learned decision boundary.

5. Algorithm / Working Process

Create positive user-item events and sampled negatives, look up branch-specific embeddings, compute elementwise product and/or MLP features, output a relevance logit, and optimize BCE or pairwise loss. At inference, score candidates—not usually the entire catalog—and rank them. A two-stage system commonly retrieves with MF/two-tower and ranks with NCF.

6. Mathematical Foundation

GMF: $\phi_{\text{GMF}} = p_u \text{ elementwise } q_i$. MLP: $z_1 = [p'_u; q'_i]$, $z_l = a_l(W_l z_{(l-1)} + b_l)$. NeuMF predicts

$\hat{y}_{ui} = \sigma(h^T[\phi_{\text{GMF}}; z_L])$.

Binary cross-entropy is $L = -\sum [y_{ui} \log \hat{y}_{ui} + (1 - y_{ui}) \log (1 - \hat{y}_{ui})]$. Pairwise alternatives optimize $-\log \sigma(s_{ui} - s_{uj})$. BCE output is not a calibrated click probability when sampled class ratios differ from serving prevalence.

7. Practical Implementation

```
import torch
from torch import nn

class NCF(nn.Module):
    def __init__(self, n_users, n_items, d=32):
        super().__init__()
        self.u = nn.Embedding(n_users, d)
        self.i = nn.Embedding(n_items, d)
        self.mlp = nn.Sequential(nn.Linear(2*d, 64), nn.ReLU(),
                                nn.Dropout(.1), nn.Linear(64, 1))

    def forward(self, users, items):
        uv, iv = self.u(users), self.i(items)
        return self.mlp(torch.cat([uv, iv], dim=-1)).squeeze(-1)
```

```
model = NCF(1000, 5000)
users = torch.tensor([1, 1, 2, 2])
items = torch.tensor([20, 90, 14, 77])
labels = torch.tensor([1., 0., 1., 0.])
logits = model(users, items)
loss = nn.functional.binary_cross_entropy_with_logits(logits, labels)
loss.backward()
print(float(loss))
```

8. Code Explanation

Separate embedding lookups encode users and items. Concatenation allows arbitrary cross-coordinate MLP interactions. The model returns logits because `binary_cross_entropy_with_logits` is numerically stable. A full NeuMF implementation adds a parallel elementwise-product branch; this minimal MLP isolates the nonlinear idea.

9. Training / Evaluation

Build temporal positive events and resample negatives by epoch, excluding known positives. Compare against popularity and MF using identical candidate sets. Tune dimension, layer widths, dropout, learning rate, negative ratio, and loss. Evaluate Recall/NDCG/MRR@K and calibration only after correcting for sampling. Watch for ID memorization and weak gains relative to latency.

10. Complexity and Cost

Per-pair cost is embedding lookup plus MLP operations, approximately $O(dh+h^2)$ depending on layers. Full-catalog scoring is expensive because nonlinear pair scoring cannot be directly ANN-indexed. GPU training helps, while inference typically scores hundreds of retrieved candidates in batches.

11. Common Use Cases

Personalized candidate scoring, media/product ranking, nonlinear CF research baselines, and second-stage ranking when metadata is limited.

12. Common Mistakes

Claiming deeper always beats dot product; comparing models with different negative samples; using random split leakage; treating sampled BCE score as probability; full-catalog online scoring; omitting a strong MF baseline.

13. Edge Cases / Limitations

Pure ID-based NCF still has cold start, needs more data and tuning than MF, and loses fast inner-product retrieval. It can memorize head users/items, is less explainable, and gains may vanish under realistic temporal evaluation.

14. Variations

- **GMF**: neuralized elementwise factor interaction; placement-important.
- **NeuMF**: GMF plus MLP fusion; canonical interview architecture.
- **Pairwise NCF**: BPR/hinge objective; use for ranking.
- **Feature-aware NCF**: adds context/content; project and production important.
- **Attention interaction**: models history-candidate relevance; advanced research.

15. Related Topics

MF is the linear baseline. Two-tower models preserve separable user/item computation for ANN retrieval. Ranking models add rich cross features and debiasing. DeepFM and factorization machines combine low-order and nonlinear sparse-feature interactions.

16. Interview Questions

1. **What limitation of MF does NCF target?** The fixed dot-product interaction function.
2. **What is GMF?** Elementwise user/item product followed by learned output weights.
3. **What is NeuMF?** Fusion of GMF and MLP interaction branches.
4. **Why sample negatives?** Unobserved pairs are enormous and ambiguous.
5. **Why can't standard NCF use simple ANN retrieval?** User and item computations interact inside the MLP.
6. **Does NCF solve cold start?** Not if it uses only ID embeddings.
7. **Which loss is common?** Pointwise BCE; pairwise BPR is another choice.
8. **Why use logits in code?** Stable combined sigmoid and BCE calculation.
9. **What baseline is mandatory?** Tuned biased MF/BPR.
10. **When is NCF justified?** When validated nonlinear gain pays for training and scoring cost.

17. Practice Tasks

Implement NeuMF; compare it with MF; vary negative ratios; test candidate-set versus full-catalog metrics; profile latency and identify whether nonlinear gain is worth it.

18. Project Ideas

Project	What it does	Stack and data	Resume value
NeuMF benchmark	Reproduces MF/GMF/MLP comparisons	PyTorch; MovieLens	Experimental rigor
Candidate scorer	NCF re-ranks ANN candidates	PyTorch/FAISS	Two-stage architecture
Contextual NCF	Adds device/time features	Lightning; KuaiRec	Feature interactions and bias analysis

19. Quick Revision

- **Key idea**: learn nonlinear user-item interaction.
- **Formula**: MLP/GMF representation to sigmoid relevance.
- **Use**: re-ranking with sufficient interaction data.

- **Metrics:** Recall/NDCG/MRR and latency.
- **Trap:** weak baseline or sampled-probability confusion.
- **One-liner:** NCF trades MF's separable dot product for a flexible but costlier scorer.

20. Final Cheat Sheet

Item	Summary
Input/output	User/item IDs/features -> relevance logit
Steps	Embed, interact, MLP, optimize, rank candidates
Hyperparameters	Dimension, layers, dropout, negatives, loss
Pros	Nonlinear expressive interactions
Cons	Cold IDs, expensive scoring, tuning/data hungry
Best use	Second-stage collaborative scoring

Two-Tower Models

1. Overview

A two-tower model independently encodes a user/query and an item into the same vector space, scoring them by dot product or cosine. Because item vectors can be precomputed and indexed, two towers are the dominant neural architecture for large-scale candidate retrieval. They combine collaborative IDs with histories, context, and content features.

2. Intuition

One network summarizes “what this user wants now”; another summarizes “what this item offers.” Training aligns compatible summaries. At serving time, compute the user once and search a warehouse of cached item vectors.

3. Prerequisites

Embeddings, sequence pooling, MLPs/encoders, contrastive learning, softmax, in-batch negatives, ANN search, feature pipelines, and temporal evaluation.

4. Core Concepts

- **User tower:** consumes ID, history, context, and query/session features.
- **Item tower:** consumes item ID and content; must be cacheable for fast retrieval.
- **Separable scoring:** $f(u)^T g(i)$ enables ANN but prevents arbitrary cross features.
- **In-batch negatives:** other batch positives become negatives, making large candidate sets cheap.
- **Hard-negative mining:** teaches distinctions among plausible candidates rather than only obvious random negatives.

5. Algorithm / Working Process

Create positive interaction pairs, encode both sides, score each user against its positive and negatives, optimize contrastive softmax, and export item vectors. Build an ANN index. Online, assemble only point-in-time user features, run the user tower, query top candidates, filter them, and send them to a richer ranker.

6. Mathematical Foundation

Let $z_u = f_{\theta}(x_u)$ and $z_i = g_{\phi}(x_i)$. Score $s(u, i) = z_u^T z_i / \tau$. In-batch sampled softmax for batch size B is

$$L = -(1/B) \sum_a \log[\exp(s(u_a, i_a)) / \sum_{(b=1)^B} \exp(s(u_a, i_b))].$$

Frequency correction can subtract $\log q(i)$ from sampled logits. L2 normalization changes maximum inner product into cosine retrieval. Retrieval quality depends on both model recall and ANN index recall.

7. Practical Implementation

```
import torch
from torch import nn

class TwoTower(nn.Module):
    def __init__(self, n_users, n_items, d=64):
        super().__init__()
        self.user = nn.Sequential(nn.Embedding(n_users, d), nn.Linear(d, d))
        self.item = nn.Sequential(nn.Embedding(n_items, d), nn.Linear(d, d))

    def forward(self, users, positive_items):
        u = nn.functional.normalize(self.user(users), dim=-1)
        v = nn.functional.normalize(self.item(positive_items), dim=-1)
        return u @ v.T # all in-batch pairs

model = TwoTower(1000, 5000)
users = torch.tensor([1, 7, 20, 33])
items = torch.tensor([50, 80, 21, 10])
logits = model(users, items) / 0.07
labels = torch.arange(len(users)) # diagonal pairs are positive
loss = nn.functional.cross_entropy(logits, labels)
loss.backward()
print(float(loss))
```

8. Code Explanation

Each tower creates a normalized vector independently. The $B \times B$ similarity matrix gives one positive on its diagonal and $B-1$ in-batch negatives per query. Cross-entropy pulls diagonal pairs above the others. A production user tower pools recent-item embeddings and context; the item tower often includes text/image/category encoders.

9. Training / Evaluation

Build pairs chronologically and prevent future history/features. Tune dimension, temperature, batch size, negative strategy, history encoder, and feature weights. Evaluate full-catalog Recall@K, NDCG, coverage, tail

recall, ANN recall, freshness, and p95 latency. Correct false negatives where multiple batch items are genuine positives for the same user.

10. Complexity and Cost

In-batch scoring costs $O(B^2d)$ but uses efficient matrix multiplication. Item encoding is offline; online user encoding plus ANN search is fast. Storage is $O(Id)$. Large batches and content encoders benefit from GPUs; serving can use CPU inference and quantized indexes.

11. Common Use Cases

YouTube-style video retrieval, product search/recommendation, ad candidate generation, job matching, creator discovery, semantic search, and multi-source feed retrieval.

12. Common Mistakes

Leaking future history; using only easy random negatives; treating all in-batch items as true negatives; forgetting item-frequency correction; training with normalized vectors but indexing unnormalized ones; measuring only model recall and not ANN recall/latency.

13. Edge Cases / Limitations

The bottleneck dot product cannot model arbitrary pairwise cross features. One user vector can blur multiple intents. Head-item dominance and stale indexes reduce discovery. New-user quality still depends on contextual/onboarding features.

14. Variations

- **DSSM-style towers:** text/query matching; canonical retrieval design.
- **History-attention tower:** candidate-independent history summary; production useful.
- **Multi-interest towers:** emit several user vectors; advanced feed retrieval.
- **Multimodal item tower:** supports new visual/text items; research/project important.
- **Distilled retrieval:** learns from a cross-encoder ranker; advanced research.

15. Related Topics

MF is a two-tower model with ID lookup and dot product. Embedding objectives supply its geometry. ANN indexes serve candidates. Ranking models then use pairwise cross features that the separable retrieval model cannot express.

16. Interview Questions

1. **Why two towers?** Independent item encoding enables precomputation and ANN retrieval.
2. **What is sacrificed?** Rich early user-item cross interactions.
3. **What are in-batch negatives?** Other positives in the batch reused as negatives.
4. **Why temperature?** It controls softmax sharpness and contrastive gradients.
5. **How do hard negatives help?** They teach fine distinctions among plausible items.
6. **How are new items handled?** Item content features produce vectors without interaction IDs.
7. **How do you prevent leakage?** Build user history/features strictly before each label time.

8. **What is ANN recall?** Fraction of exact model top items recovered by the approximate index.
9. **Why frequency correction?** Sampled item prevalence distorts logits toward frequent items.
10. **Retrieval versus ranking?** Retrieval maximizes recall cheaply; ranking optimizes final order richly.

17. Practice Tasks

Add mean-pooled history to the user tower; implement in-batch loss; mine hard negatives; build a FAISS index; measure exact versus ANN Recall@100 and latency.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Video retriever	Retrieves from watch histories	PyTorch/FAISS; MovieLens	Production candidate generation
Job two-tower	Matches candidate and job features	Transformers/ANN; job posts	Semantic cold-start matching
Multimodal shop	Indexes text/image catalog	CLIP/PyTorch; Amazon products	Multimodal retrieval

19. Quick Revision

- **Key idea:** independently encode query and item for ANN retrieval.
- **Formula:** contrastive softmax over dot products.
- **Use:** million-item candidate generation.
- **Metrics:** full-catalog Recall@K, ANN recall, latency.
- **Trap:** false/easy negatives and train-index mismatch.
- **One-liner:** two towers buy scalable retrieval by keeping user and item computation separable.

20. Final Cheat Sheet

Item	Summary
Input/output	User/item features -> shared-space vectors/candidates
Steps	Encode pairs, contrast, cache items, ANN query
Hyperparameters	Dimension, temperature, batch, negatives, history
Pros	Feature-aware, scalable, cold-item capable
Cons	Limited cross features, index complexity
Best use	Large-catalog candidate retrieval

Ranking Models

1. Overview

Ranking models assign relevance or utility scores to retrieved candidates and determine their final order. They combine user, item, context, cross, and system features to predict clicks, watch time, purchases, satisfaction, or multi-objective value. Gradient-boosted trees, logistic models, Wide & Deep, DeepFM, and transformer rankers are common.

2. Intuition

Retrieval is a recruiter collecting plausible resumes; ranking is the hiring panel inspecting each candidate with richer evidence. It can ask whether this specific user-item pair fits the current time, device, price, and recent intent.

3. Prerequisites

Candidate retrieval, classification/regression, learning-to-rank, feature engineering, calibration, position/exposure bias, delayed labels, and online experimentation.

4. Core Concepts

- **Pointwise ranking:** predict an outcome per item; simple and scalable but only indirectly optimizes order.
- **Pairwise ranking:** learn that relevant i should outrank j ; aligns with ordering.
- **Listwise ranking:** optimize a whole slate/list approximation; closest to ranking metrics but complex.
- **Cross features:** user-item-category-price/history compatibility unavailable to simple retrieval.
- **Multi-objective value:** blend click, conversion, retention, quality, diversity, and constraints.
- **Bias correction:** clicks depend on prior exposure and position, not only relevance.

5. Algorithm / Working Process

Log candidate impressions and positions, join point-in-time features and delayed outcomes, construct training examples, train/calibrate a pointwise/pairwise/listwise model, and evaluate chronologically. Online, retrieve candidates, compute consistent features, batch-score them, apply constraints/re-ranking, and return a slate.

6. Mathematical Foundation

Pointwise BCE: $L = -\sum [y \log \sigma(s) + (1-y) \log (1-\sigma(s))]$. Pairwise RankNet/BPR: $L = -\log \sigma(s_{ui} - s_{uj})$. A listwise softmax loss is

$L = -\sum_i P_y(i) \log P_s(i)$, where $P_s(i) = \exp(s_i) / \sum_j \exp(s_j)$.

LambdaRank scales pair gradients by the change $|\Delta \text{NDCG}|$ caused by swapping items. Inverse propensity scoring estimates unbiased risk with $\sum_o L_i / p(\text{exposure}_i)$, but very small propensities require clipping to control variance. A multi-task score might be $a P(\text{click}) + b P(\text{purchase}) + c P(\text{hide})$.

7. Practical Implementation

```
import numpy as np
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score
```

```
# Example point-in-time features: retrieval score, user-category affinity,  
# price gap, item popularity, hour; y is click after an impression.  
rng = np.random.default_rng(42)  
X = rng.normal(size=(2000, 5))  
y = (1.5*X[:, 0] + .8*X[:, 1] - .4*X[:, 2] + rng.normal(size=2000) >  
1).astype(int)  
cut = 1600 # stand-in for a chronological cutoff  
ranker = HistGradientBoostingClassifier(max_depth=5, learning_rate=.05)  
ranker.fit(X[:cut], y[:cut])  
scores = ranker.predict_proba(X[cut:][:, 1])  
print("validation AUC:", roc_auc_score(y[cut:], scores))
```

8. Code Explanation

The model is a pointwise ranker: predicted click probabilities induce an ordering. Histogram gradient boosting handles nonlinearities and feature interactions strongly on tabular data. In a real pipeline, group rows by request for ranking metrics, split by timestamp rather than row index alone, and ensure every feature was available at impression time.

9. Training / Evaluation

Train on impressions, not only positives, and join conversion labels after a maturity window. Use time splits and request-level groups. Offline metrics include NDCG, MAP, MRR, Recall, AUC, log loss, and calibration; slice by position, cohort, and catalog head/tail. Online A/B tests measure CTR, conversion, watch time, retention, diversity, complaints, and latency. Tune depth/layers, regularization, sampling, loss, objective weights, and calibration.

10. Complexity and Cost

Tree inference is approximately $O(TD)$ for T trees/depth D per candidate; neural models depend on layer and sequence sizes. Cost multiplies by candidate count, so expensive cross-encoders rank only a small final set. Feature fetching often dominates p95 latency and requires caches/batching.

11. Common Use Cases

Home feeds, search results, ads, marketplace listings, notifications, video rails, job matching, and final ordering after multi-source retrieval.

12. Common Mistakes

Training only on clicked items; leaking post-impression features; random splitting repeated requests; optimizing AUC while product cares about top positions; ignoring position/exposure bias; mixing retrieval and ranking scores without calibration; failing training-serving feature parity.

13. Edge Cases / Limitations

Logged labels reflect the old policy and unshown items lack outcomes. Delayed rewards, sparse purchases, feedback loops, multi-stakeholder trade-offs, and changing feature distributions complicate learning. The

highest independent item scores may form a redundant or unfair slate.

14. Variations

- **GBDT/LightGBM LambdaMART:** strong tabular list ranking; placement/project-essential.
- **Wide & Deep/DeepFM/DCN:** sparse IDs plus nonlinear crosses; industry-important.
- **Sequence/cross-encoder ranker:** rich history-candidate interaction; research/large-scale use.
- **Multi-task ranking:** shares click/conversion/watch representations; production-important.
- **Slate re-ranking:** diversity, fairness, constraints; advanced production topic.

15. Related Topics

Two towers supply high-recall candidates. Bandits explore and correct policy dependence. Evaluation metrics define ranking objectives. Calibration, causal inference, and counterfactual learning address biased logged feedback.

16. Interview Questions

1. **Pointwise vs pairwise vs listwise?** Per-item labels, item-order comparisons, or whole-list objectives.
2. **Why not rank the whole catalog with a deep ranker?** Pair-specific computation is too expensive.
3. **Why train on impressions?** Non-clicked exposed items form meaningful negatives.
4. **What is position bias?** Higher positions receive more clicks independent of relevance.
5. **AUC vs NDCG?** AUC measures global pair ordering; NDCG emphasizes top ranked positions.
6. **What is LambdaRank?** Pairwise gradients weighted by metric change, commonly Delta-NDCG.
7. **How handle multiple objectives?** Multi-task predictions plus constrained/weighted value and online validation.
8. **What is feature leakage here?** Any feature computed after the recommendation/label time.
9. **Why calibrate?** Downstream bidding/value blending needs meaningful probabilities.
10. **How validate success?** Offline temporal metrics followed by guarded online experiments.

17. Practice Tasks

Train pointwise GBDT and pairwise ranker; compute NDCG by request; simulate position bias; diagnose a leaked popularity feature; add diversity re-ranking and measure relevance trade-off.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Learning-to-rank lab	Compares point/pair/list losses	LightGBM/PyTorch; MSLR	Ranking fundamentals
Feed ranker	Retrieves then multi-task ranks	FAISS/PyTorch; MIND	End-to-end recommender architecture
Fair marketplace ranker	Balances relevance and exposure	XGBoost/OR tools; synthetic	Multi-objective system design

19. Quick Revision

- **Key idea:** richly score retrieved candidates for final order.
- **Formula:** BCE, pairwise logistic, or listwise loss.
- **Use:** final precision and business objectives.
- **Metrics:** NDCG/MAP/MRR, calibration, online outcomes.
- **Trap:** biased impressions and leaked features.
- **One-liner:** retrieval finds plausible items; ranking spends compute to choose the best order.

20. Final Cheat Sheet

Item	Summary
Input/output	Candidate features -> score/final slate
Steps	Log, join, train, calibrate, score, constrain
Hyperparameters	Loss, depth/layers, sampling, objective weights
Pros	Rich features and objectives; high precision
Cons	Bias, latency, feature complexity
Best use	Final-stage personalized ordering

Cold Start Problem

1. Overview

Cold start is the inability to personalize an entity or situation with insufficient interaction history. It includes new users, new items, new systems/domains, and context cold start. It matters because every recommender faces new traffic and catalog inventory; a model that performs well only for established IDs can fail at product launch and growth boundaries.

2. Intuition

A new waiter does not know a first-time customer's preferences, and diners do not know a newly added dish. The restaurant uses priors, menu descriptions, a few questions, and controlled tasting—exactly the combination of popularity, content, onboarding, and exploration used in recommenders.

3. Prerequisites

Popularity/content/CF models, priors and uncertainty, embeddings, metadata pipelines, contextual features, exploration-exploitation, and cohort-based evaluation.

4. Core Concepts

- **New-user cold start:** no behavioral profile; solve with context, onboarding, session behavior, or safe popularity.
- **New-item cold start:** no interaction factor; solve with content/category/creator features and exploration.

- **System/domain cold start:** little data anywhere; use rules, pretrained representations, transfer, and deliberate data collection.
- **Warm-up threshold:** route entities based on evidence amount rather than one global model.
- **Preference elicitation:** ask a small, diverse set of high-information questions/items.

5. Algorithm / Working Process

Detect coldness using interaction count/age/uncertainty, choose an evidence-appropriate strategy, collect metadata and context, blend cold and warm scores smoothly, explore promising uncertain choices, and transition toward collaborative models as events arrive. Output is still top K , but routing and blending depend on entity maturity.

6. Mathematical Foundation

A simple empirical-Bayes blend is

$s(u,i) = \alpha_n s_{\text{personal}}(u,i) + (1-\alpha_n)s_{\text{prior}}(i)$, with $\alpha_n = n/(n+\beta)$.

For a new item, blend content and collaborative scores by interaction count. An upper-confidence exploration score is $\hat{\mu}_i + c \sqrt{\log t/(n_i+1)}$. Information-oriented onboarding can choose questions maximizing expected entropy reduction $H(\theta) - E[H(\theta | \text{answer})]$.

7. Practical Implementation

```
def cold_start_score(personal, content, popularity,
                    user_events: int, item_events: int,
                    user_prior=10, item_prior=20):
    """Smooth routing avoids a discontinuity at an arbitrary warm threshold."""
    user_w = user_events / (user_events + user_prior)
    item_w = item_events / (item_events + item_prior)
    item_ready = item_w * personal + (1 - item_w) * content
    return user_w * item_ready + (1 - user_w) * popularity

print(cold_start_score(personal=.9, content=.7, popularity=.4,
                      user_events=2, item_events=0))
```

8. Code Explanation

The weights rise gradually with evidence. A completely new item relies on content; a completely new user relies on popularity. As both warm up, the collaborative/personal score dominates. Production systems learn or validate these blending curves by cohort and preserve exploration slots.

9. Training / Evaluation

Create explicit cold cohorts: zero-history users, users with 1/3/5 events, new items by launch time, and warm entities. Hide pre-cutoff interactions to simulate onboarding honestly. Report Recall/NDCG, coverage, time-to-first-value, activation/retention, new-item exposure, regret, and fairness. Never average cold users into a warm majority and declare success.

10. Complexity and Cost

Popularity and metadata fallbacks are cheap; content encoders add offline GPU cost, and online feature inference must meet latency. Maintaining multiple routes increases operational complexity more than algorithmic complexity. Blending is $O(1)$ per candidate.

11. Common Use Cases

First app session, newly listed products/jobs/videos, marketplace seller onboarding, regional launch, anonymous traffic, and seasonal catalogs.

12. Common Mistakes

Using random splits that make test items look warm; assigning unknown IDs to one meaningless embedding; applying the same strategy to new users and new items; abrupt thresholds; asking too many onboarding questions; never exploring new inventory.

13. Edge Cases / Limitations

Metadata can be missing, deceptive, or unable to capture quality. New users may skip onboarding or change intent. Exploration can hurt short-term metrics, and sparse domains may never become fully warm. Privacy rules may restrict demographic/context use.

14. Variations

- **Popularity/context fallback:** quickest user solution; placement-essential.
- **Content-to-CF hybrid:** new-item solution; project/production-essential.
- **Active preference elicitation:** optimize onboarding questions; research-important.
- **Meta-learning/transfer:** adapt from other domains; research-focused.
- **Bandit exploration:** safely collect evidence; production-important.

15. Related Topics

Content filtering provides new-item vectors, two towers encode unseen features, bandits explore uncertain entities, session models personalize anonymous users, and Bayesian smoothing supplies stable priors.

16. Interview Questions

1. **What types of cold start exist?** User, item, system/domain, and new context.
2. **Why doesn't MF handle a new ID?** No learned row exists for that entity.
3. **Best new-item strategy?** Content/creator/category representation plus controlled exploration.
4. **Best new-user strategy?** Context, onboarding, session signals, and popularity fallback.
5. **Why smooth blending?** Evidence quality changes gradually, not at a magic count.
6. **How do you evaluate cold start?** Dedicated history-count and item-age cohorts.
7. **Can demographics solve it?** They provide a prior but introduce privacy/fairness risks.
8. **How do you choose onboarding items?** Diverse, popular, informative items that separate tastes.
9. **What is time-to-warm?** Events/time needed to reach reliable personalized quality.
10. **Why explore?** Without exposure, new items cannot obtain evidence.

17. Practice Tasks

Create zero/one/five-history test cohorts; implement blend routing; compare popularity with content for new items; optimize a five-item onboarding slate; debug ID leakage in a random split.

18. Project Ideas

Project	What it does	Stack and data	Resume value
New-user onboarding	Selects diverse taste questions	Streamlit/PyTorch; MovieLens	Product + information gain
Launch-aware marketplace	Blends content/CF by maturity	FastAPI/FAISS; Amazon	Real cold-item serving
Cross-domain starter	Transfers book to movie tastes	Transformers; public ratings	Research-style adaptation

19. Quick Revision

- **Key idea:** route by available evidence and collect signal safely.
- **Formula:** evidence-weighted personal/prior blend.
- **Use:** every new entity and launch.
- **Metrics:** cold Recall/NDCG, coverage, activation, time-to-warm.
- **Trap:** random splits hiding cold failure.
- **One-liner:** cold start is an evidence problem, solved by priors, features, elicitation, and exploration.

20. Final Cheat Sheet

Item	Summary
Input/output	Sparse/new entity + features/context -> fallback/blended top-K
Steps	Detect, route, blend, explore, warm up
Hyperparameters	Warm priors, blend curve, exploration rate
Pros	Robust launch/new-traffic experience
Cons	Weaker signal, policy and metadata dependence
Best use	New users/items/domains

Evaluation Metrics

1. Overview

Recommendation evaluation measures whether relevant items appear near the top, how the system behaves across the catalog/users, and whether it improves real product outcomes. No single metric is sufficient: offline

relevance, beyond-accuracy qualities, online business/user outcomes, latency, and guardrails must be considered together.

2. Intuition

A recommendation list is like a search page: finding the right item anywhere is not enough; it should appear early. Meanwhile, showing ten near-identical correct choices may be less useful than a diverse slate, and clicks may not equal long-term satisfaction.

3. Prerequisites

Confusion counts, ranked lists, logarithms, probability calibration, temporal splits, sampling bias, hypothesis testing, and A/B experiments.

4. Core Concepts

- **Precision@K:** fraction of shown items relevant; values quality of limited slots.
- **Recall@K/Hit Rate:** fraction of relevant items recovered / whether any is found; retrieval-oriented.
- **MRR:** reciprocal rank of first relevant item; appropriate for one desired answer.
- **AP/MAP:** rewards every relevant item and their ordering.
- **DCG/ND CG:** graded relevance with top-heavy logarithmic discount.
- **Coverage/diversity/novelty:** measure catalog reach and user discovery beyond accuracy.
- **Online metrics:** CTR, conversion, watch time, retention, satisfaction, and guardrails reveal causal product impact via experiments.

5. Algorithm / Working Process

Define the recommendation event and relevance label, choose a point-in-time split, generate candidates against the full eligible catalog, filter training-seen items consistently, compute per-user/request metrics, aggregate with slices and uncertainty, compare baselines, then run an online experiment before launch.

6. Mathematical Foundation

For relevant set G_u and list L_u^K :

- $Precision@K = |G_u \cap L_u^K| / K$
- $Recall@K = |G_u \cap L_u^K| / |G_u|$
- $MRR = (1/U) \sum_u 1/rank_u$
- $DCG@K = \sum_{r=1}^K (2^{rel_r} - 1) / \log_2(r+1)$ and $ND CG = DCG / IDC G$
- $AP@K = (1/\min(|G_u|, K)) \sum_r Precision@r * rel_r$; MAP averages AP.
- Intra-list diversity can be $(2/(K(K-1))) \sum_{(i < j)} (1 - sim(i, j))$.

Sampled-negative metrics are optimistic and not comparable across different sampled candidate sets. Confidence intervals may use user-level bootstrap. A/B uplift is $mean(metric_treatment) - mean(metric_control)$ with variance computed at the randomization unit.

7. Practical Implementation

```
import numpy as np

def ranking_metrics(recommended, relevant, k=10):
    recs, truth = recommended[:k], set(relevant)
    hits = np.array([item in truth for item in recs], dtype=float)
    precision = hits.sum() / k
    recall = hits.sum() / len(truth) if truth else 0.0
    dcg = sum(hit / np.log2(rank + 1) for rank, hit in enumerate(hits, 1))
    ideal_hits = min(len(truth), k)
    idcg = sum(1 / np.log2(rank + 1) for rank in range(1, ideal_hits + 1))
    ndcg = dcg / idcg if idcg else 0.0
    mrr = 1 / (np.flatnonzero(hits)[0] + 1) if hits.any() else 0.0
    return {"precision": precision, "recall": recall, "ndcg": ndcg, "mrr": mrr}

print(ranking_metrics([4, 8, 2, 1], {2, 7}, k=4))
```

8. Code Explanation

The binary `hits` vector records relevance by rank. Precision and recall count hits with different denominators. DCG discounts later hits, IDCG normalizes by the best possible list for this user, and MRR finds only the first hit. Production evaluation also averages over requests and reports confidence intervals and cohorts.

9. Training / Evaluation

Use leave-last-one-out or global chronological splits only when they match serving. Avoid future interactions in histories/features. Full-catalog evaluation is preferred; if sampling is necessary, freeze the set and label results clearly. Tune on validation, touch test once, report popularity and random baselines, and follow offline results with A/B testing. Monitor relevance, diversity, coverage, fairness, latency, and long-term retention.

10. Complexity and Cost

Metric calculation after ranked lists is $\mathcal{O}(UK)$, but generating full-catalog rankings dominates. Exact top-K scoring can be $\mathcal{O}(UI)$; ANN and batched GPU scoring reduce it. User-level bootstrap multiplies evaluation cost but is embarrassingly parallel.

11. Common Use Cases

Model selection, retrieval/ranker diagnosis, launch gates, A/B analysis, drift dashboards, subgroup audits, and regression tests.

12. Common Mistakes

Random temporal leakage; reporting sampled HR@10 as full-catalog quality; averaging only users with hits; using precision when each user has one test positive without explaining it; optimizing clicks alone; tuning on the test set; ignoring eligibility and seen-item policies.

13. Edge Cases / Limitations

Offline relevance observes only exposed/chosen items, so unclicked does not always mean irrelevant. Metrics conflict: diversity may reduce short-term precision. Empty ground-truth users need a declared policy. Online novelty can initially reduce clicks while improving long-term value.

14. Variations

- **Graded NDCG:** ratings/value labels; placement-essential.
- **Recall/Hit Rate:** candidate retrieval; production-essential.
- **Calibration:** alignment of recommendation category mix or predicted probabilities; project-important.
- **Counterfactual/off-policy metrics:** correct logged-policy bias; research-important.
- **Long-term metrics:** retention/lifetime value; product-critical but delayed/noisy.

15. Related Topics

Ranking losses approximate these discontinuous metrics. Bandits require regret and off-policy evaluation. Cold-start/session systems need cohort-specific measurement. Statistical testing separates noise from online uplift.

16. Interview Questions

1. **Precision@K vs Recall@K?** Slot purity versus fraction of all relevant items recovered.
2. **Why NDCG?** It supports graded relevance and discounts lower ranks.
3. **When use MRR?** When the first useful result dominates, such as navigation/search.
4. **Hit Rate vs Recall with one positive?** They are equivalent per user.
5. **Why is sampled evaluation optimistic?** The model avoids competing against most catalog items.
6. **Why chronological split?** Deployment predicts future behavior from past information.
7. **What is coverage?** Fraction of catalog or users receiving recommendations.
8. **Offline vs online?** Offline is cheap/correlational; randomized online testing estimates product impact.
9. **What unit should bootstrap use?** The independent unit, often user rather than impression.
10. **Can AUC evaluate ranking?** It measures global pair ordering but underweights the top of the list.

17. Practice Tasks

Implement MAP/NDCG; compare sampled with full-catalog Recall; bootstrap confidence intervals; find leakage in a random split; measure relevance-diversity trade-offs under MMR re-ranking.

18. Project Ideas

Project	What it does	Stack and data	Resume value
RecSys evaluator	Reusable temporal metric suite	Python/Polars; MovieLens	Evaluation rigor
A/B simulator	Power and uplift analysis	scipy/Streamlit; synthetic	Experimentation skill
Fairness dashboard	Cohort relevance/exposure audit	Pandas/Plotly; MIND	Responsible recommendation

19. Quick Revision

- **Key idea:** assess rank relevance, catalog/user quality, and causal outcomes.
- **Formula:** NDCG normalizes discounted gain by ideal gain.
- **Use:** model selection and launch validation.
- **Metrics:** Recall, NDCG, MAP/MRR, coverage/diversity, online KPIs.
- **Trap:** leakage and sampled-candidate inflation.
- **One-liner:** a recommender is not evaluated by one accuracy number but by ranked utility under realistic exposure.

20. Final Cheat Sheet

Item	Summary
Input/output	Ranked lists + relevance/outcomes -> metric suite
Steps	Temporal split, rank, per-user score, aggregate/slice, A/B
Hyperparameters	K, relevance rule, candidate policy, discount
Pros	Directly measures product ranking behavior
Cons	Biased labels; offline-online gap
Best use	Every recommendation experiment

Session-Based Recommendation

1. Overview

Session-based recommendation predicts the next or most useful item from the ordered events in a current session, often without a stable user identity. It captures short-term intent and preference drift. E-commerce, news, music, video, and anonymous browsing use Markov chains, session-KNN, RNNs/GRUs, convolution, and transformers.

2. Intuition

A person who usually buys electronics may currently browse hiking boots. The current trail of clicks is more informative than their lifetime average. A session model reads that trail like words in a sentence and predicts what comes next.

3. Prerequisites

Sequence modeling, Markov assumptions, RNN/GRU/attention, causal masks, embeddings, next-item objectives, padding/masking, and chronological splits.

4. Core Concepts

- **Sessionization:** group events by user and inactivity gap; the boundary changes training examples.
- **Order/recency:** recent events often reveal current intent more strongly.
- **Next-item prediction:** every prefix can predict its following item.
- **Anonymous state:** model a sequence rather than a permanent ID.

- **Intent shift:** attention or multiple-interest states prevent old clicks from dominating.

5. Algorithm / Working Process

Sort events by time, split sessions after an inactivity threshold, create (prefix,next item) examples without crossing boundaries, encode items and positions, process prefixes with transition counts/GRU/transformer, score candidate items, train with softmax or sampled loss, and recommend online as each event extends the session.

6. Mathematical Foundation

A first-order Markov model estimates $P(i_t=j|i_{(t-1)}=i)=C_{ij}/\sum_k C_{ik}$. A GRU updates hidden state via reset/update gates and predicts $P(i_t|h_{(t-1)})=\text{softmax}(W h_{(t-1)})$. A causal transformer computes

$$\text{Attention}(Q,K,V)=\text{softmax}(QK^T/\sqrt{d}+M)V,$$

where mask M blocks future positions. Next-item cross-entropy is $L=-\sum_t \log P(i_t | i_1, \dots, i_{(t-1)})$. Sampled softmax/BPR handles large catalogs.

7. Practical Implementation

```
import torch
from torch import nn

class GRU4RecMini(nn.Module):
    def __init__(self, n_items, d=64):
        super().__init__()
        self.embed = nn.Embedding(n_items, d, padding_idx=0)
        self.gru = nn.GRU(d, d, batch_first=True)
        self.output = nn.Linear(d, n_items)
    def forward(self, sessions, lengths):
        x = self.embed(sessions)
        packed = nn.utils.rnn.pack_padded_sequence(
            x, lengths.cpu(), batch_first=True, enforce_sorted=False)
        _, hidden = self.gru(packed)
        return self.output(hidden[-1])

model = GRU4RecMini(n_items=1000)
sessions = torch.tensor([[4, 8, 2], [7, 3, 0]])
lengths = torch.tensor([3, 2])
targets = torch.tensor([9, 6])
loss = nn.functional.cross_entropy(model(sessions, lengths), targets)
loss.backward()
print(float(loss))
```

8. Code Explanation

Padding ID 0 is ignored by its embedding. Packed sequences prevent the GRU from treating padding as behavior. The final hidden state summarizes each prefix and the output layer scores all items. For huge catalogs, use sampled losses or a retrieval head instead of a dense output layer.

9. Training / Evaluation

Split by session end time and never split a future suffix into training. Evaluate next-item Recall/MRR/NDCG@K by prefix length, session length, device, and anonymous/new-item cohorts. Tune inactivity gap, maximum length, embedding/hidden size, dropout, learning rate, negatives, and recency. Compare with last-item popularity, transition, and session-KNN baselines.

10. Complexity and Cost

Markov lookup is cheap. GRU cost is $O(Ld^2)$ per sequence; transformer attention is $O(L^2d)$ time and $O(L^2)$ attention memory. Dense catalog output is $O(Id)$. GPUs help training; online state caching and short windows keep inference fast.

11. Common Use Cases

Next product/page/video/song, cart completion, anonymous homepage adaptation, short-form feed, search-query refinement, and conversational recommendations.

12. Common Mistakes

Randomly splitting events from one session; generating examples across session boundaries; padding without masks/lengths; evaluating only long sessions; using future items in bidirectional attention; ignoring repeat-item policy and timestamp gaps.

13. Edge Cases / Limitations

The first event has little context, noisy clicks mislead, extremely long sessions contain multiple intents, and new items still need content. Fixed timeouts may split one intent or merge unrelated visits. Pure session models discard useful long-term preferences.

14. Variations

- **Markov/transition model:** strong simple baseline; placement-essential.
- **Session-KNN:** retrieves similar recent sessions; project-important.
- **GRU4Rec/NARM:** recurrent sequence and attention; interview/research important.
- **SASRec/BERT4Rec:** causal/masked transformer sequences; research and industry important.
- **Long + short-term hybrid:** combines user history with session intent; production-preferred.

15. Related Topics

Item embeddings encode sequence proximity, graph session models treat each session as a graph, two towers retrieve with a session encoder, and bandits adapt recommendations from immediate feedback.

16. Interview Questions

1. **Why session models?** Current ordered behavior reveals short-term intent and supports anonymous users.
2. **How define a session?** User/device events separated by an inactivity threshold or explicit boundary.
3. **What is the label?** Usually the next item after each prefix.

4. **GRU vs transformer?** GRU is linear in length; transformer captures long-range dependencies but costs quadratic attention.
5. **Why causal masking?** Prevent future items from leaking into next-item representation.
6. **What is a strong baseline?** Last-item transition or session-KNN.
7. **How handle first click?** Item-to-item/content/popularity/context fallback.
8. **How combine long-term taste?** Fuse user embedding with session state or use gated attention.
9. **Why MRR?** The first correct next item's position often matters most.
10. **How handle multiple intents?** Shorten windows, use attention, intent clusters, or multi-interest states.

17. Practice Tasks

Sessionize event logs; build a transition baseline; train GRU4Rec; test performance by prefix length; debug a causal leakage/padding error; extend with long-term user embedding.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Next-click engine	Compares Markov, KNN, GRU	PyTorch; Retailrocket	Sequential baseline discipline
Music session DJ	Continues short listening sessions	SASRec/FAISS; Last.fm	Transformer retrieval
Intent-shift monitor	Visualizes state change in sessions	Plotly/PyTorch; Yoochoose	Model analysis

19. Quick Revision

- **Key idea:** predict from ordered short-term context.
- **Formula:** next-item cross-entropy under causal history.
- **Use:** anonymous and rapidly changing intent.
- **Metrics:** Recall/MRR/NDCG@K by prefix length.
- **Trap:** session/future leakage.
- **One-liner:** session recommendation models what the user wants now, not only who they usually are.

20. Final Cheat Sheet

Item	Summary
Input/output	Ordered session prefix -> next/top-K items
Steps	Sessionize, prefix-label, encode, predict, update state
Hyperparameters	Gap, length, dimension, layers, negatives
Pros	Intent-aware, anonymous-friendly
Cons	Sparse initial context, sequence cost, noisy boundaries
Best use	Next-item and anonymous personalization

Graph-Based Recommendation

1. Overview

Graph-based recommendation models users, items, and possibly creators/categories as nodes connected by interactions or relations. It exploits multi-hop structure: a user connects to items, which connect to similar users and further items. Random walks, Personalized PageRank, graph embeddings, and graph neural networks (GNNs) are major approaches.

2. Intuition

Imagine spreading a small amount of preference energy from a user across a graph. It flows to consumed items, then to other users and related items. Nodes receiving more relevant energy become candidates. GNNs learn how much and what information to pass along these paths.

3. Prerequisites

Graphs and sparse adjacency matrices, random walks/Markov chains, embeddings, message passing, normalization, graph sampling, and collaborative filtering.

4. Core Concepts

- **Bipartite graph:** user-item edges represent behavior; no direct same-type edges are required.
- **Heterogeneous graph:** adds entity/relation types such as brand, tag, social follow, or knowledge graph facts.
- **Message passing:** aggregate neighbor representations to capture collaborative context.
- **Over-smoothing:** too many layers make node representations indistinguishable.
- **Edge weighting/time:** event type, strength, and recency determine graph signal.

5. Algorithm / Working Process

Construct a leakage-safe graph from past events, initialize node embeddings/features, propagate normalized neighbor messages for several layers or run personalized walks, combine layer representations, optimize edge-ranking/contrastive loss, and retrieve top items from final embeddings. Update graph/index as interactions arrive.

6. Mathematical Foundation

Personalized PageRank satisfies $\mathbf{p}_{i,u} = \alpha \mathbf{e}_u + (1-\alpha) \mathbf{P}^T \mathbf{p}_{i,u}$, with transition matrix $\mathbf{P}$. A GCN layer is

$$\mathbf{E}^{(l+1)} = \text{sigma}(\mathbf{D}_{\text{tilde}}^{-1/2} \mathbf{A}_{\text{tilde}} \mathbf{D}_{\text{tilde}}^{-1/2} \mathbf{E}^{(l)} \mathbf{W}^{(l)}).$$

LightGCN removes transformations/nonlinearities for collaborative graphs:

$$\mathbf{e}_u^{(l+1)} = \sum_{i \in N(u)} \mathbf{e}_i^{(l)} / \sqrt{(|N(u)| |N(i)|)}, \text{ and similarly for items. Final } \mathbf{e} = \sum_1 \alpha_{l-1} \mathbf{e}^{(l)}, \text{ score } \mathbf{e}_u^T \mathbf{e}_i, \text{ typically trained with BPR plus L2 regularization.}$$

7. Practical Implementation

```
import torch

# Tiny normalized user-item propagation without a GNN library.
n_users, n_items, d = 3, 4, 8
edges = torch.tensor([[0, 0], [0, 1], [1, 1], [1, 2], [2, 3]])
U = torch.randn(n_users, d, requires_grad=True)
V = torch.randn(n_items, d, requires_grad=True)
user_deg = torch.bincount(edges[:, 0], minlength=n_users).clamp_min(1)
item_deg = torch.bincount(edges[:, 1], minlength=n_items).clamp_min(1)
weight = 1 / torch.sqrt(user_deg[edges[:, 0]] * item_deg[edges[:, 1]])

next_U = torch.zeros_like(U).index_add(0, edges[:, 0], V[edges[:, 1]] * weight[:, None])
next_V = torch.zeros_like(V).index_add(0, edges[:, 1], U[edges[:, 0]] * weight[:, None])
final_U, final_V = (U + next_U) / 2, (V + next_V) / 2
scores = final_U @ final_V.T
print(scores.shape) # [users, items]
```

8. Code Explanation

Degree-normalized edge weights stop high-degree nodes from overwhelming aggregation. `index_add` sums neighboring item messages into users and user messages into items. Averaging initial and propagated embeddings retains both identity and one-hop collaborative context, matching the core LightGCN idea. Training would apply BPR to positive and sampled-negative scores.

9. Training / Evaluation

Build graph edges before the cutoff only; sampling subgraphs must preserve labels and degree logic. Tune layers, dimension, layer weights, edge/time weights, dropout, negatives, and regularization. Evaluate Recall/NDCG, coverage, tail performance, cold cohorts, scalability, and inference freshness. Compare with MF: graph gains must justify graph construction cost.

10. Complexity and Cost

Full message passing per layer is approximately $O(Ed)$ time and node embeddings cost $O((U+I)d)$. Multi-relational models add parameters and edges. Neighbor sampling or mini-batch graph training is necessary at scale. Final inner-product retrieval uses ANN; graph refresh and distributed storage dominate operations.

11. Common Use Cases

Social recommendations, related products, knowledge-aware movies, fraud-resistant marketplaces, creator/content discovery, Pinterest-style visual graphs, and session graphs.

12. Common Mistakes

Adding a GNN when plain MF on the same bipartite graph suffices; leaking validation edges during propagation; using excessive layers; ignoring degree/popularity effects; negative-sampling known edges; building an undirected graph when relation direction matters.

13. Edge Cases / Limitations

Isolated nodes remain cold, hubs dominate walks/messages, noisy edges spread errors, and over-smoothing/over-squashing hurt deep propagation. Graph training and fresh updates are operationally complex. Explanations based on paths may still be misleading.

14. Variations

- **Random walk/PageRank:** no supervised training, explainable paths; placement-important.
- **Node2Vec/DeepWalk:** random-walk embeddings; project-important.
- **NGCF/LightGCN:** collaborative GNNs; research/interview-important.
- **Knowledge-graph recommendation:** typed semantic relations; advanced research.
- **Session graph neural networks:** transitions within a session; sequence applications.

15. Related Topics

MF factorizes graph connectivity; LightGCN repeatedly smooths its embeddings. Item-item CF is a projected co-occurrence graph. Embeddings represent nodes, while two towers/ANN serve final graph-derived vectors.

16. Interview Questions

1. **Why a graph for recommendation?** Interactions naturally form relational structures with useful multi-hop signal.
2. **What does message passing do?** Aggregates neighbor representations into a contextual node vector.
3. **Why degree normalization?** Controls scale and hub dominance.
4. **What is LightGCN?** Simplified collaborative propagation without feature transforms/nonlinearities.
5. **Why can one layer help?** It brings directly interacted item/user information into the node.
6. **What is over-smoothing?** Deep propagation makes node embeddings too similar.
7. **How does it differ from MF?** It explicitly propagates higher-order neighborhood signals before scoring.
8. **How handle new nodes?** Node features/inductive GNNs or content fallback; pure ID transduction cannot.
9. **Main leakage risk?** Allowing held-out edges into the training graph/message paths.
10. **When is a GNN not worth it?** When the graph is only simple bipartite data and tuned MF performs equally.

17. Practice Tasks

Implement personalized PageRank; add a second LightGCN layer; compare MF/LightGCN; measure hub bias; debug graph leakage; incorporate time-decayed edge weights.

18. Project Ideas

Project	What it does	Stack and data	Resume value
LightGCN benchmark	Compares graph CF with BPR-MF	PyTorch Geometric; MovieLens	GNN rigor and ablation

Project	What it does	Stack and data	Resume value
Knowledge movie graph	Uses actor/director/genre paths	Neo4j/PyG; MovieLens+IMDb	Heterogeneous graph skills
Session graph shop	Predicts next click via item graph	DGL; Yoochoose	Dynamic sequence graphs

19. Quick Revision

- **Key idea:** propagate preference over interaction relations.
- **Formula:** normalized neighbor aggregation plus dot-product/BPR.
- **Use:** meaningful multi-hop or heterogeneous structure.
- **Metrics:** Recall/NDCG, tail coverage, latency.
- **Trap:** held-out edges in the graph and unnecessary GNN complexity.
- **One-liner:** graph recommenders enrich embeddings with neighbors-of-neighbors before ranking.

20. Final Cheat Sheet

Item	Summary
Input/output	Nodes/edges/features -> node embeddings/top-K
Steps	Build, initialize, propagate/walk, optimize, retrieve
Hyperparameters	Layers, dimension, normalization, edge weights, negatives
Pros	Multi-hop and heterogeneous relation modeling
Cons	Complex, hub-biased, cold isolated nodes
Best use	Rich interaction/knowledge/social graphs

Bandit-Based Recommendation

1. Overview

Bandit-based recommendation chooses actions while learning from their rewards, balancing exploitation of known good items with exploration of uncertain ones. A multi-armed bandit ignores context; a contextual bandit conditions on user/item/request features. It is useful for fast-changing content, new items, notifications, ads, and online adaptation.

2. Intuition

A restaurant mostly serves dishes customers already love but occasionally offers a promising new special. Never exploring prevents learning; exploring randomly too often wastes meals. Bandit algorithms attach value to both expected reward and uncertainty.

3. Prerequisites

Probability distributions, expected value, confidence intervals/Bayesian updating, online learning, regret, contextual features, randomized experiments, and propensity logging.

4. Core Concepts

- **Arm/action:** item or policy choice; large catalogs often use a retriever to define available arms.
- **Reward:** click, purchase, dwell, or delayed value; design determines behavior.
- **Exploration-exploitation:** choose known reward versus gather information.
- **Regret:** cumulative reward lost relative to the best policy/action.
- **Propensity:** probability the logging policy chose an action; required for unbiased off-policy evaluation.
- **Non-stationarity:** preferences and item quality change, requiring decay/windows.

5. Algorithm / Working Process

At each request, observe context and eligible actions, estimate reward and uncertainty, select an action/slate by a policy, log context/action/propensity, observe immediate or delayed reward, and update estimates. Guardrails constrain unsafe exploration. Batch rankers commonly generate candidates while a bandit explores among them or tunes slate positions.

6. Mathematical Foundation

Regret after T rounds is $R_T = \sum_t (\mu^* - \mu(a_t))$. Epsilon-greedy chooses the current best with probability $1 - \epsilon$ and explores otherwise. UCB1 chooses

$$a_t = \operatorname{argmax}_a [\hat{\mu}_a + c \sqrt{\log t / n_a}].$$

For Bernoulli Thompson sampling, prior $\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$; after reward $r \in \{0, 1\}$, update $\alpha_a += r$, $\beta_a += 1 - r$, sample each θ_a , and pick the maximum. A linear contextual bandit assumes $E[r|x, a] = x_a^T \theta$; LinUCB score is $x_a^T \hat{\theta} + \alpha \sqrt{x_a^T A^{-1} x_a}$.

Inverse propensity scoring evaluates target policy π from logged policy π_0 :

$$\hat{V}_{\text{IPS}} = (1/N) \sum_t r_t \pi(a_t | x_t) / \pi_0(a_t | x_t).$$

7. Practical Implementation

```
import numpy as np

class ThompsonBandit:
    def __init__(self, n_arms, seed=42):
        self.alpha = np.ones(n_arms) # successes + prior
        self.beta = np.ones(n_arms) # failures + prior
        self.rng = np.random.default_rng(seed)

    def choose(self):
        samples = self.rng.beta(self.alpha, self.beta)
        return int(np.argmax(samples))

    def update(self, arm, reward):
        self.alpha[arm] += reward
```

```
self.beta[arm] += 1 - reward

bandit = ThompsonBandit(3)
true_ctr = [0.03, 0.05, 0.04]          # simulator only
for _ in range(5000):
    arm = bandit.choose()
    reward = bandit.rng.random() < true_ctr[arm]
    bandit.update(arm, int(reward))
print("estimated CTR:", bandit.alpha / (bandit.alpha + bandit.beta))
```

8. Code Explanation

Each arm has a Beta posterior over Bernoulli click rate. Sampling naturally explores arms with high uncertainty and exploits arms with high means. Successes/failures update conjugate posterior parameters. This simple model is non-contextual and single-slot; production policies add context, slate effects, delayed rewards, and strict logging.

9. Training / Evaluation

Before live traffic, use replay/simulation cautiously and off-policy estimators on randomized logs. Online, monitor cumulative reward/regret proxies, CTR/conversion/retention, exploration exposure, subgroup harm, and guardrails. Tune exploration `epsilon/c/alpha`, priors, decay window, reward definition, and update cadence. Roll out gradually against a stable control.

10. Complexity and Cost

UCB/Thompson selection is $O(A)$ for A arms; large catalogs require candidate restriction or structured embeddings. LinUCB updates may cost $O(d^2)$ with maintained inverses. Online state, delayed event joins, and reliable low-latency updates are the main engineering costs; GPU is rarely necessary for simple bandits.

11. Common Use Cases

News headlines, notifications, ads, new-item exposure, homepage module selection, promotion choice, adaptive onboarding, and exploration over ranker candidates.

12. Common Mistakes

Not logging propensities; defining reward as click while harming retention; exploring unsafe/ineligible actions; assuming stationarity; evaluating a new policy directly on deterministic logs; treating unobserved rewards as zero; updating on delayed/misattributed conversions.

13. Edge Cases / Limitations

Rewards may be delayed, censored, sparse, or confounded by position. Multiple simultaneous items interact, violating independent-arm assumptions. Exploration has real user/business cost. New policies cannot be evaluated where the logging policy has zero support.

14. Variations

- **Epsilon-greedy:** simplest baseline; placement-essential.
- **UCB:** optimism under uncertainty; interview-essential.
- **Thompson sampling:** posterior probability matching; project/production important.
- **Contextual/LinUCB/neural bandits:** personalize exploration; advanced roles.
- **Combinatorial/slate bandits:** choose interacting lists; research-important.
- **Non-stationary bandits:** windows/discounted updates; news and trends.

15. Related Topics

Reinforcement learning handles long-horizon state transitions; bandits optimize immediate/contextual decisions. Ranking supplies candidate reward estimates, cold start motivates exploration, and causal/off-policy evaluation uses propensity weighting.

16. Interview Questions

1. **What is exploration-exploitation?** Learning uncertain rewards while earning from current knowledge.
2. **Bandit vs supervised ranking?** Bandits choose data-collecting actions; supervised models learn from fixed logs.
3. **What is regret?** Cumulative gap from an optimal comparator.
4. **How does UCB explore?** Adds an uncertainty bonus to estimated reward.
5. **How does Thompson sampling explore?** Samples plausible reward parameters and acts greedily for the sample.
6. **What is a contextual bandit?** Reward/action choice depends on observed request features.
7. **Why log propensity?** It enables importance weighting and policy auditing.
8. **What is the support problem?** Offline evaluation cannot assess actions the logging policy never chose.
9. **Bandit vs RL?** Bandits lack modeled long-term state/action consequences.
10. **How make exploration safe?** Candidate constraints, conservative policies, caps, guardrails, and staged rollout.

17. Practice Tasks

Simulate epsilon-greedy/UCB/Thompson; plot cumulative regret; add drifting CTR; implement LinUCB; estimate a target policy with IPS; debug a missing-propensity logging pipeline.

18. Project Ideas

Project	What it does	Stack and data	Resume value
Bandit playground	Compares policies under drift	NumPy/Streamlit; simulator	Clear exploration intuition
News headline selector	Contextual click optimization	Vowpal Wabbit/FastAPI; MIND	Online learning architecture
Safe catalog explorer	Gives new items bounded exposure	Python/Kafka; synthetic retail	Cold-start and guardrails

19. Quick Revision

- **Key idea:** learn rewards through controlled online decisions.
- **Formula:** UCB mean plus uncertainty; Thompson posterior sampling.
- **Use:** adaptive exploration and fast-changing choices.
- **Metrics:** reward, regret, off-policy value, guardrails.
- **Trap:** missing propensities and bad reward design.
- **One-liner:** bandits make recommendation a learn-while-serving problem.

20. Final Cheat Sheet

Item	Summary
Input/output	Context, actions, reward history -> chosen action/slate
Steps	Estimate, explore/exploit, log propensity, observe, update
Hyperparameters	Exploration strength, priors, decay/window, reward
Pros	Online adaptation, principled exploration
Cons	Exploration cost, delayed/bias issues, support limits
Best use	Dynamic content and cold-item learning

End-to-End Interview Synthesis

A production answer should distinguish stages and objectives:

1. **Candidate generation:** popularity, item-item, matrix factorization, graph embeddings, or a two-tower model retrieves hundreds from millions with high Recall@K.
2. **Ranking:** a GBDT or neural ranker uses rich cross/context features to optimize NDCG and calibrated business outcomes.
3. **Re-ranking:** constraints, freshness, diversity, fairness, and deduplication construct the final slate.
4. **Cold-start routing:** content, context, onboarding, and popularity cover entities without factors.
5. **Learning loop:** impression/action/propensity/outcome logs create leakage-safe training data; bandits add controlled exploration.
6. **Evaluation:** chronological full-catalog offline evaluation is followed by a guarded online experiment and long-term monitoring.

In an interview, always clarify the interaction type, catalog scale, latency budget, new-entity rate, objective, and what exposure logs exist before choosing an algorithm.